

INDUSTRIAL PROJECT: UTIL LABS GENERIC TEST JIG FRAMEWORK

A Dissertation

by

JACOB JACOBUS GREEFF

209329085

Submitted in partial fulfillment of the requirements for the degree

BACHELOR TECHNOLOGIAE

in

Electrical Engineering

Department of Electrical Engineering

Faculty of Engineering and the Built Environment

TSHWANE UNIVERSITY OF TECHNOLOGY

Supervisor: R Aylward

Co-Supervisor: J Olwagen

Tshwane

July 2011

DECLARATION

“I hereby declare that this dissertation submitted for the degree Bachelor Technologiae at the Tshwane University of Technology, is my own original unaided work and has not previously been submitted to any other institution or higher education. I further declare that all sources cited or quoted are indicated and acknowledge by means of a comprehensive list of references”.

JJ Greeff

ACKNOWLEDGEMENTS

I would like to sincerely thank those who made this thesis possible, specifically my wife Carol who gave me the moral support I required to complete it in time. My two supervisors Mr. Ron Aylward and Mr. Jan Olwagen who helped me with the research material and guiding me to the end result. I also would like to make special reference to Mr. Joe Paul who is the CEO of Util Labs who was willing to allow me to take a chance on the test equipment and let me have free reign creatively in the production department, as well as Mr. Cobus Greyling who helped out with the mechanical design and assembly for the mechanical jig components.

Without the cooperation of the people above, this thesis would not have been possible.

ABSTRACT

AUTHOR: Jacob Jacobus Greeff
DEGREE: Btech: Electrical Engineering
TITLE: Industrial Project: Util Labs Generic Test Jig Framework
UNIVERSITY: Tshwane University of Technology
FACULTY: Faculty of Engineering and the Built Environment
SUPERVISOR: Mr. Ron Aylward
CO-SUPERVISOR: Mr. Jan Olwagen
DATE: July 2011
KEYWORDS: Manufacturing, Testing, Production,

a test jig is a device used to test some component or assembly. In this report, the unit to test is a populated circuit board, and the test jig an electronic devices using a bed of nails interface to the unit under test. This bed of nails is made up of a number of spring loaded pins protruding through some isolating medium that are connected via suitable wiring to the heart of the test jig which allows measurements to be taken from and stimuli to be injected into the unit under test. Using this interface, the correct functionality of the unit under test is verified by comparing the measured values from the board with a defined set of test values with a suitable tolerance. Since the test jig is in many ways a reference point determining which products pass and which products fail in a manufacturing run, it must never be in question as to whether the jig is reliable, accurate or repeatable.

Manufacturing in Util Labs relies on a set of test jigs that have proven to be not only unreliable and difficult to maintain, but due to the lack of documentation and source code, have proven difficult to keep up to date and expand. As the Load Limiter project has progressed, different devices have been added to the product spectrum of the company and since different companies at different times have been contracted to create test jigs for each of these products, they are all different and suffer from different weaknesses. This project aims to

rectify this situation by creating a solid test jig framework, that can be expanded to suit the requirements of the specific products being tested, but is also suited to the style of manufacturing employed at Util labs (using multiple contract manufacturers). Since this framework will be used in the future to create all test equipment for circuit boards at Util Labs, new test jigs based on this framework should be as easy as possible to create.

Table of Contents

DECLARATION	i
ACKNOWLEDGEMENTS.....	ii
ABSTRACT.....	iii
GLOSSARY OF TERMS.....	xii
CHAPTER 1:.....	1
1.1 INTRODUCTION AND BACKGROUND.....	1
1.2 PROBLEM STATEMENT.....	2
1.2.1 Initial cost of test jig and Scalability.....	2
1.2.2 Maintenance turn around time.....	2
1.2.3 Design integration (pre and post).....	2
1.2.4 Development turn around time.....	3
1.2.5 Traceability.....	3
1.2.6 Documentation.....	4
1.3 USER REQUIREMENTS SPECIFICATIONS.....	5
1.3.1 Stakeholders.....	5
1.3.2 User Requirements Specification.....	6
1.4 PROJECT OBJECTIVES.....	9
1.5 SCOPE.....	11
1.6 TIMELINE AND BUDGET.....	13
1.6.1 Timeline.....	14
1.6.2 Budget.....	16
1.7 CONCLUSION.....	17
CHAPTER 2.....	18
2.1 INTRODUCTION.....	18
2.2 LITERATURE SURVEY.....	19
2.3 PROPOSED PRACTICAL DESIGN.....	23
2.3.1 Test Jig Pcb.....	23
2.3.2 Test Jig Pc.....	24
2.3.4 Bed of Nails.....	25

2.4 CONSTRAINTS.....	27
2.5 TEMPLATING.....	29
2.6 GENERAL USE CASE.....	30
2.7 PRODUCT SPECIFICATION.....	32
2.8 CONCLUSIONS.....	34
CHAPTER 3.....	35
3.1 INTRODUCTION.....	35
3.2 THE PROTOTYPE.....	36
3.2.1 Prototype Characteristics.....	36
3.2.2 Detailed Design Description.....	37
3.2.2.1 Power Supplies.....	37
3.2.2.2 Controllers.....	39
3.2.2.3 Isolated Bi-Directional Ports.....	44
3.2.2.4 Analog Measurement.....	46
3.2.2.5 UUT Comms.....	47
3.2.2.6 Storage.....	48
3.2.3 The Test Jig Creation Process	49
3.2.4 Design Review.....	50
3.3 THE SECOND ITERATION – FIRST PRACTICAL IMPLEMENTATION.....	55
3.3.1 Iteration 2 Characteristics.....	56
3.3.2 Test Jig Creation Process.....	57
3.3.3 Design Review – Changes Made To Iteration 2 To Create Iteration 3.....	60
3.4 THE THIRD ITERATION.....	63
3.4.1 Iteration 3 Characteristics.....	64
3.4.2 Test Jig Creation Process.....	65
3.4.3 Design Review – Changes Made To Iteration 3 To Create Iteration 4.....	68
3.5 THE FOURTH ITERATION.....	72
3.5.1 Iteration 4 Characteristics.....	73
3.5.2 Test Jig Creation Process.....	75
3.5.3 Design Review – Changes Made to Iteration 4 to Create Iteration 5.....	79
3.6 THE FIFTH ITERATION.....	81

3.6.1	Iteration 5 Characteristics.....	82
3.6.2	Detailed Design Description.....	83
3.6.2.1	Power Supply.....	83
3.6.2.2	ICoupler.....	89
3.6.2.3	Output Driver	91
3.6.2.4	Controller.....	92
3.6.2.5	Light Sensor.....	93
3.6.2.6	Impedance Measurement And Power Test.....	94
3.6.3	Firmware.....	96
3.6.3.1	Inputs and Outputs.....	97
3.6.3.2	Fast Inputs.....	98
3.6.3.3	ADC.....	99
3.6.3.4	Class Summary.....	101
3.6.4	Test Jig Creation Process.....	103
3.7	THE JIG MECHANICS.....	110
3.8	TEST PC JAVA SOFTWARE ARCHITECTURE.....	112
3.8.1	Creating the Test Boards.....	114
3.8.2	Creating the Implementation.....	115
3.8.3	Add The Implementation To The Detector.....	117
3.10	CONCLUSION.....	118
CHAPTER 4		119
4.1	INTRODUCTION.....	119
4.2	RESULTS – USER REQUIREMENTS SPECIFICATION.....	120
4.3	RESULTS – PRODUCT SPECIFICATION	122
4.3	CONCLUSION	124
CHAPTER 5		125
5.1	INTRODUCTON.....	125
5.2	CONCLUSIONS.....	126
5.3	RECOMMENDATIONS.....	128
5.4	PROPOSED FURTHER WORK.....	129

APPENDIX A.....	I
Generic Test Point Placement.....	I
Test coverage.....	I
Test Point Checklist.....	II
Specifications on placement.....	IV
APPENDIX B.1	VI
APPENDIX B.2.....	XXI
APPENDIX C	XXII
APPENDIX D	XXXIII
APPENDIX E.....	XLV
APPENDIX F.1	XLXVI
APPENDIX F.2	LXXXI
APPENDIX G.....	LXXXII
System architecture overview.....	LXXXII
EOS Architecture.....	LXXXV
REFERENCES.....	XCI

Illustration Index

Illustration 1: Project Plan.....	13
Illustration 2: Floating Probe System.....	19
Illustration 3: JTAG programmer.....	21
Illustration 4: Proposed Test Jig System.....	23
Illustration 5: Bed of Nails layers.....	25
Illustration 6: Different types of test pins.....	27
Illustration 7: The standard test flow.....	31
Illustration 8: Isolated +3.3V and +5V power supply set.....	38
Illustration 9: Master Processor schematic.....	39
Illustration 10: Slave Processor Schematic.....	40
Illustration 11: Jtag Interface.....	41
Illustration 12: Inter Processor Bus.....	42
Illustration 13: RS485 driver chip schematic.....	43
Illustration 14: Connector To The LCD.....	44
Illustration 15: Isolated Input Schematic.....	44
Illustration 16: Isolated Output Schematic.....	45
Illustration 17: Isolated Analog Front End.....	46
Illustration 18: UUT Comms Circuit.....	47
Illustration 19: Storage on SD card and Flash.....	48
Illustration 20: Test Jig Design Process.....	49
Illustration 21: Partially populated prototype board during test.....	50
Illustration 22: Test Jig Creation Process.....	57
Illustration 23: Creating The Test Jig Stack In Iteration 2.....	57
Illustration 24: Test Jig Stack Mock Up.....	59
Illustration 25: Test Jig Generic Base Mock Up.....	59
Illustration 26: Iteration 2 Implementation Bottom.....	61
Illustration 27: Iteration 2 Implementation Top.....	61
Illustration 28: Test Jig Stack Creation In Iteration 3.....	65
Illustration 29: Guide board showing a cut-out made for a dense number of test points.....	68

Illustration 30: Generic Test Jig Rev C before population.....	69
Illustration 31: Iteration 4 Test Jig Stack Creation.....	75
Illustration 32: Stubs On ICoupler On Stopper Template.....	77
Illustration 33: Optional Switch Mode Power Supply On Stopper Template.....	77
Illustration 34: Fitting Iteration 4 Boards On the Mechanical Base Plate.....	79
Illustration 35: Mutually Exclusive Placement of R44 and D6.....	84
Illustration 36: 3.3V Switch Mode Power Supply.....	86
Illustration 37: The 12V And 5V On Board Power Supplies.....	87
Illustration 38: ICoupler Schematic.....	89
Illustration 39: ICoupler CMOS Transformer technology.....	89
Illustration 40: Vias Added And Copper Over Pads To Increase Heat Sync.....	90
Illustration 41: The 40 pin Interface Connector And Output Driver.....	91
Illustration 42: Schematic For One Channel Of The ULN2003A driver.....	91
Illustration 43: STM32F103 Microprocessor Generic Test Board Controller.....	92
Illustration 44: Light sensor.....	93
Illustration 45: Impedance Measurement Circuit.....	94
Illustration 46: Input and Output Actions (Only Relevant Methods and Members Shown).....	97
Illustration 47: Pulse Test Actions (Only relevant methods and members).....	98
Illustration 48: ADC Test Actions (Only relevant methods and members).....	100
Illustration 49: Iteration 5 Test Jig Stack Creation.....	103
Illustration 50: Board To Test Created As .brd File.....	104
Illustration 51: Close Up Of Heat Sync And Stubs On ICoupler On Stopper Template.....	107
Illustration 52: Outline Board Created Using Positioner.....	109
Illustration 53: Generic Test Boards and Test Jig Stack Bolted To Lifting Plate.....	111
Illustration 54: The Java Architecture Overview.....	112
Illustration 55: Test Board Specific Implementations.....	114
Illustration 56: Test Jig Implementation In The Framework.....	115
Illustration 57: Adding The Implementation To The Detector.....	117
Illustration 58: Virtual Message Bus Showing An UPWARDS Routed Message.....	LXXXII
Illustration 59: What Each Device Connected To The Virtual Message Bus "Sees".....	LXXXIV
Illustration 60: EOS Operating System Modules Interactions.....	LXXXV

Index of Tables

Table 1: Manufacturing test solutions.....	22
Table 2: Hardware designs associated with the prototype.....	36
Table 3: comparing JTAG and serial bootloader programming on the test jig.....	53
Table 4: Hardware designs associated with iteration 2.....	56
Table 5: Hardware designs associated with iteration 3.....	64
Table 6: P1 and P2 mappings on temp-*boardname*.brd.....	66
Table 7: Maximum UUT Board Size For Iteration 4.....	75
Table 8: Hardware designs associated with iteration 5.....	82
Table 9: Components To Place For External vs On Board Power Supply.....	84
Table 10: Actions Performed By The Generic Test Jig Controller.....	92
Table 11: Results - User Requirements Specifications.....	121
Table 12: Functional Specifications Results.....	123
Table 13: Electrical Specifications Results.....	124
Table 14: Interface Specifications.....	124
Table 15: Test Point Check List.....	IV

GLOSSARY OF TERMS

TUT:	Tshwane University of Technology. The home page for the university can be located at: www.tut.ac.za
URS:	User Requirements Specification
MCU:	Measurement and Control Unit, the core measurement product in the Util Labs family of products.
Test Jig:	An electronic device used to test a unit or assembly that is being manufactured or tested against some known reference(s). In the scope of this project, the test jig will be a bed of nails type test device (or similar) used to test manufactured circuit boards.
Test Pin:	a spring loaded pin, generally made from copper with a gold plating that acts as the interface point on the bed of nails between the circuit board being tested, and the test jig. ^[11] The Test pins used throughout this project are from Ingun ^[12] .
Bed Of Nails:	An interface to a circuit board used on the test jig that consists of a number of spring loaded pins that, when actuated will connect to the circuit board to be tested so that reference stimuli can be injected into it and measurements of electronic signals can be made on the board.
JTAG:	The acronym for the Joint Test Action Group, that is now synonymous with the debug interface described in the standard IEEE 1149.1-1990. This port is used to debug, test and program various processors and other programmable devices like FPGAs and DSPs. ^[5]
iCoupler:	iCoupler technology is a registered trademark of Analog Devices ^[24] that defines a way of getting signals across an electrical isolation barrier. It is one of the core technologies in this project that allows electrical isolation between the unit under and the control pc of the test jig.
ADC:	Analog to Digital conversion. The act of measuring an analog voltage signal and converting it to a representative digital signal using appropriate electronic hardware. In the scope of this project, analog to digital

conversion is achieved either by using a dedicated ADC chip (AD7918)^[21] or using the on board peripheral of the ARM7 STM32F103 micro processor^[19].

LDR: Light Dependant Resistor. A resistor that changes its resistance based on the amount of light that is incident on it at any point in time. In the scope of this project, LDRs are the main technique of measuring if an optical component (like a lamp or LED) on the unit under test is functional or not. ^[27]

Unit Under Test: The device that is presently being tested by a test jig. In the scope of this project, the unit under test will be the circuit board that has been placed on the bed of nails and is being tested.

Test Jig Stack: The test jig stack is the set of base, guide, stopper and outline pcbs that make up a single bed of nails for a specific type of unit to be tested.

Cost per Test: Cost per test is a value used in production to factor the cost of testing equipment and testing time into the cost of a manufactured product.

BOM: Bill Of Materials that contains all of the components and sub-systems required to manufacture a system or sub-system.

GUI: Graphical User Interface

Kicad: An open source suite of EDA (Electronic Design Automation) software used to capture schematics and create pcb artwork. Kicad is composed of 5 main components:

- Kicad Project manager
- Eeschema schematic capture
- PCBnew pcb artwork creator
- Cvpcb used to match schematic components to artwork
- Gerbview gerber viewer.

Kicad is used extensively in this project in all schematic and pcb work.

The home page for the Kicad project is

<http://iut-tice.ujf-grenoble.fr/kicad/>

Eclipse: Eclipse is a generic open source IDE (Integrated Development Environment) used to write, build, run and debug code written in a number of languages, but optimised specifically Java. In this project, all code written in Java has been written and tested using the Eclipse IDE. The home page of the eclipse project is:

<http://www.eclipse.org/>

Crossworks: Rowley Crossworks is a commercial IDE used for specifically writing, building and testing code for embedded applications in C/C++. All code written in this project on the STM32F103 series of processors was done in Crossworks. The home page for Rowley is:

<http://www.rowley.co.uk/>

CHAPTER 1:

1.1 INTRODUCTION AND BACKGROUND

Util Labs is a fairly new company that is rapidly entrenching itself as a large player in the energy sector in South Africa by offering a holistic approach to metering, energy balancing and smart grid load and appliance control. With a national footprint of 30 000 metering points, with planned expansion in 2011 to 80 000 metering points, the company aims to be the de facto standard for load control in South Africa, as well as keeping an eye on the international market by forming agreements with companies in various countries, including Brazil, Poland, India, Italy etc.

All products in the Util Labs range, form part of a larger system that can be expanded to meet the requirements of the customer (in the case of South Africa, the customer is Eskom or the local municipalities that supply electricity to all people in the country). These products are all developed in house and manufactured in South Africa. The style of manufacture used is one of contract manufacturing, where Util Labs is responsible for supplying a contract manufacturer with Datapacks, assembly instructions, components and test equipment needed to assemble the product.

This system works very well, but there are a number of problems that have been faced in the past production runs due to the fact that early on in the project, all the design of test equipment to be used was outsourced to an external company, and the test equipment received has proven to be expensive, rigid, unreliable and not scalable.

It is with an eye on solving the production problems faced in the manufacturing of product that this project was embarked on.

1.2 PROBLEM STATEMENT

As mentioned in the introduction, the testing phase of the manufacturing of product at external contract manufacturers has been identified as a problem. The following issues have been raised and need to be addressed by this project:

1.2.1 Initial cost of test jig and Scalability

One of the problems faced is that there is a very large initial cost to getting a test jig made, especially around the machining of the bed of nails itself. The problem with this is that there is no chance to have small production runs that are tested to the same level as large production runs as it is not economically feasible to buy a test jig when only 1000 units are created. This limits the ability to prototype new products in large volume, as only a small number can be created when they need to be tested by hand. Additionally, the test jigs are fast enough as single test jigs for medium sized production runs, but when production gets ramped, the jigs are a bottleneck, as there is no way to speed up the throughput on a single jig, and buying additional test jigs will be as expensive as the original.

1.2.2 Maintenance turn around time

Due to the cost of the jigs, there are no hot-swappable backups available – this means that when there is a breakage on the jig during a manufacturing run, it causes a long line stoppage while the test jig is debugged and fixed. To overcome this, a set of hot-swappable jigs need to be created in the framework for each PC board with the goal of dropping line stoppage time to below 20 minutes.

1.2.3 Design integration (pre and post)

Currently the process to create a test jig involves capturing the design gerber files and marking on the schematic where the test pins should go and then sending this to an external company. The engineer at the external company uses the hand markings on the schematics and gerber files to program a CNC drilling machine to create the bed of nails front face. Once the front face is milled and drilled, the test pin holders are inserted and the pins are wired up to a number of relay controllers, SCADA devices, power supplies etc. depending on the requirements of the board to be tested. Once the hardware is created, a custom application is written for the PC to control the jig.

In addition to being laborious, this process has so many steps in it, all with human intervention, that the chances of a bug creeping into the eventual bed of nails are quite large. When this process is followed from start to finish all at once it is not much of a problem to have so many steps, but later on in the design cycle of products when changes need to be made to the bed of nails (for example the moving of a test point, or the addition of a test point) It makes it very difficult to make any modifications to the test jig as all of the steps need to be followed again due to the heavily propriety nature of the design (each test jig has a different internal layout, and a different PC application). This is a costly process due to the level of interaction involved, but also adds the risk of damaging the bed of nails, as to make the changes the current test jig needs to be disassembled, modified and reassembled. Often due to the timelines involved because of line stoppages, quick and dirty (generally undocumented) changes are made to the existing jigs which cause problems in the long run.

The main problems identified here are the lack of integration of process linking the design of the product and the design and manufacture of its corresponding test jig, and the large differences between test jigs.

1.2.4 Development turn around time

As an add on point to the previous point, due to the back and forth interaction between the company and it's test jig manufacturer, the development of a new test jig can be a slow process that can take a couple of months to develop. This time is spent on the test jig once the product requiring the jig has already been designed, as it is only once the product is completed that the full test regime for it can be set up. This therefor increases the time it takes between the development department releasing a design for manufacture and the final product arriving in stores, by the length of time it takes to design and manufacture the test jig.

1.2.5 Traceability

Currently, all the test jigs used respond with a simple pass/fail interface that puts the onus on the operator of the jig to sort the boards that failed to go to the different rework stations, or in the case of boards that cannot be repaired to write them off. This is a flawed process as it allows for human error to creep in and places too much responsibility in the process of sorting the boards on the operator.

1.2.6 Documentation

Since all of the test jigs currently in production are different, there is no central document archive for them. Each jig has differing levels of design, implementation, installation and operating documentation ranging from none at all to some. In addition to the updating of the processes used to creating test jigs, one of the secondary goals of this project is to get a full set of documentation (design document, manufacturing document, installation document and operations manual) for each of the test jigs created.

1.3 USER REQUIREMENTS SPECIFICATIONS

1.3.1 Stakeholders

The stakeholders of the framework proposed in this project are not restricted to the individual operators of the test jigs, so it is important to list the interested parties here before outlining the user requirements specification:

- **The production manager.**

The production manager is concerned with keeping failure rates and efficiency on the production line optimized. As such, it is in his/her interest to have reliable test equipment that can not only identify individual failed units, but also allows for the detection of failure trends throughout the production process that will point to possible inefficiencies in the process.

- **The operations manager.**

The operations manager is concerned with the flow of components and orders to the factory, as well as the flow of product to the customer from the factory. The benefit that the operations manager will get from this project is directly related to the cost per test and level of scalability that the project can achieve.

- **The operator.**

The operator will be the the person that actually handles the test jig and tests the individual PC boards that will form part of the manufactured product. The benefit that the operator should get from the project is a simpler interface and also having the process of sorting failed/passed product simplified.

- **The Test Engineer.**

The test engineer is responsible for overseeing the test equipment and processes followed on all of the products manufactured at a factory. The test engineer will also be responsible for fault finding a failed test jig. The benefit the Test engineer will get from this project is hot swappable jigs, with a simpler interface, that should minimize downtime.

- **The development engineer**

The development engineer is responsible for the designing of products (in this case in the Util Labs family of products). His/Her interaction with the production department is such that when he/she has completed the design, It is handed over to

the production department so that a set of test jigs can be created for the individual PC boards/units as well as the product as a whole. The benefit the developer/engineer should see from this project is a much higher level of integration where he/she has a greater input into the testing process and can see the testing of his/her product done much faster.

- **Integration Testing Department**

The Integration testing department is responsible for continually testing the full system of Util Labs products whenever a new hardware/software version is released to ensure that integrity is maintained across the full product line. The Integration testing department is the gatekeeper between the development department and the production department. By supplying the department with a set of test jigs that can be filled with products that have a known failure, the system can be tested to see what the reaction is in different failure modes.

- **Quality officer.**

The quality officer is responsible for ensuring that the processes followed in manufacturing the product conforms to the highest quality standards, in the case of Util Labs using the principles in the ISO9000 framework [1]. The benefit that the quality officer should gain from the project is the traceability in failures that should allow him/her to optimise out all problems in the process with the help of the operations and production managers, as well as a set of documentation that should conform to the quality requirements set out in the ISO9000 framework.

With these users in mind, the user requirements specification can be set up.

1.3.2 User Requirements Specification

The user requirement specifications need to meet with the following points:

- **System Description**

The result of this project should be a framework that will allow the creation of test jigs that can accept a manufactured PC board, test it against a predetermined test sequence, and indicate to the operator whether the tested PC board passed or failed based on the test criteria.

- **Cost**

The framework put into place should be such that the test jigs generated should be significantly cheaper to manufacture than the current set of test jigs. This does not take into account the development time taken to create the jig, just the physical cost of components and time to manufacture it. The goal price is set to < R20 000 per full testing station compared to the current cost of between R20 000 and R100 000 per test station.

- **Scalability**

Scalability in this context is the ability to add more test equipment to test units in parallel and thereby ramp up production. The way in which this is achieved is not as important as the ability to test a number of units all at once. Additionally the cost of the test jig must be such that a number of test jigs can be bought for a specified budget and used in parallel. The cost of this scalability should be such that if production is required to double throughput (and by extension double the testing capabilities of the factory), that the cost of the operation would be less than R2 on each unit for a 10 000 unit production run. In other words, doubling the testing station should cost less than R20 000.

- **Design Flexibility**

As much hardware and software as possible should be re-used and shared between test jigs to bring down development time as well as increasing the reliability of individual components in the test jigs. Code and hardware should therefor be designed to be as flexible as possible so that it can be applied to a number of different situations. The design flexibility should be such that for a moderately sized test jig with:

- 6 analog inputs
- 10 digital inputs
- 10 switched outputs
- 4 optical inputs
- 2 pulsed inputs

The design time would be less than 1 month for both the hardware and software (including testing and documentation).

- **Integration**

The integration of the test jig design into the design process of new products is necessary to minimise the time required to create a test jig for a new product. The changing of a current test jig must also be easy enough and cheap enough to make changes on test jigs that have already been designed viable. The integration should be such that to add/move/remove a test point from a pre-existing test jig design to take into account new changes to it should take less than 2 man days.

- **Maintenance**

The maintenance of the test jig should be such that it doesn't cause line stoppages of more than half an hour. The cost should also be such that hot swappable test jigs are available, and the price of a secondary set of hot swappable jigs should be less than R2 on each unit for a 10 000 unit production run. In other words, a hot swappable testing station should cost less than R20 000.

- **Support traceability**

In order to increase traceability on the line of faults in the production process, the test jig must also make provision for a system that will remove the responsibility to sort the different failed units from the operator and increase automation in the process. The final product should implement a system where each individual test on the test jig is regarded as a separate failure mode, with its own recommended course of action.

1.4 PROJECT OBJECTIVES

The main design objective in this project is to replace the current family of test jigs used at Util Labs with a new generic family of test jigs that will support the style of manufacture as well as the wide array of different products manufactured and sold by the company.

Secondary objectives are as follows:

- **Design Integration**

The author hopes to achieve a greater level of design integration with the design process as implemented in Util Labs. This integration should allow test points to be added to a design either during the design phase, or to take a previous design and add test points to it and then create a test jig for the board directly from the design files. The objective would be to achieve a full design turnaround time of one month or less on a test jig. The test jig framework of creating new test jigs should be designed in such a way as to support this objective.

- **Simplicity**

The framework should generate test jigs that are simple enough to use that with only simple button or lever interfaces an operator can go through the entire test sequence for a single unit, as well as allowing for a simple way to sort failed units so that they may be reworked or thrashed depending on the nature of the failure, to support the quality department in goal of producing the highest possible quality of product.

- **Scalability**

The framework should create test jigs that are scalable by virtue of their very low cost, as well as the ability to reuse as many components as possible. This same cost requirement would support the creation of hot swappable backup test jigs to be made available to the factory manufacturing on behalf of Util Labs.

- **Reliability**

The test jigs created in the frame work should be reliable enough to do a run of at least 10 000 units without failing, and in the event of failure, should require no more work to swap out or repair than can be done in half an hour to get the production line back up to speed again.

- **Ease of installation**

The set up of the test jigs created in the framework should be simple enough that the product being manufactured can be changed, and it should take no more than 1 hour to re-set up the next set up jigs for the production run of the new product.

1.5 SCOPE

The scope of this project is the generation of a test jig framework, as well as any test jigs that are created within the framework during the development of it. This includes:

- The hardware that will run the actual tests, injecting into and measuring signals from the UUT.
- The hardware platforms of the individual beds of nails that hold the pins map to the test points on the UUT.
- Any firmware created on the hardware and bed of nails platforms including tasks, gateways and any device drivers coded specifically for the purpose of the generic test jig framework.
- The processes involved in creating the beds of nails from the design files of a product to be tested.
- The application code required to support the generic test jigs, including any classes required to communicate to the test jigs, collecting and interpreting the test values and controlling the flow of testing.
- The porting of the EOS operating system to support the test jig firmware.
- The expansion of the existing Java swing (swing is part of the core set of Java test jig technologies, used in this project to create a graphical user interface to the operator) [2] framework to support the interface to the operator.
- The expansion of the message catalog and supporting firmware and software to support the test jig specific messages on the standard message bus.
- The expansion of the label printing capabilities of the Java swing framework.

Some parts of the design of this framework however do NOT form part of this project, owing to the fact that they had already been in place before the commencement of this project, and although the author was involved in their development in many instances they form part of the current code base of Util Labs:

- The EOS base operating system, including its file system, message bus implementation and existing base hardware interfaces.
- The Java swing GUI framework itself.

- The common message bus structure implementation that allows all devices in the Util Labs family of products to communicate to the server with the same message catalog.
- the JTAG Class interfacing to the command line interface of OpenOCD, other than the small changes made in support of the project.
- The Base mechanical design specific to the generic test jig framework. (Although this work is done specifically for the framework, it is done in collaboration with the mechanical engineering arm of Util Labs, and as such all mechanical designs done in Solidworks are done by them, and not the author.)

1.6 TIMELINE AND BUDGET

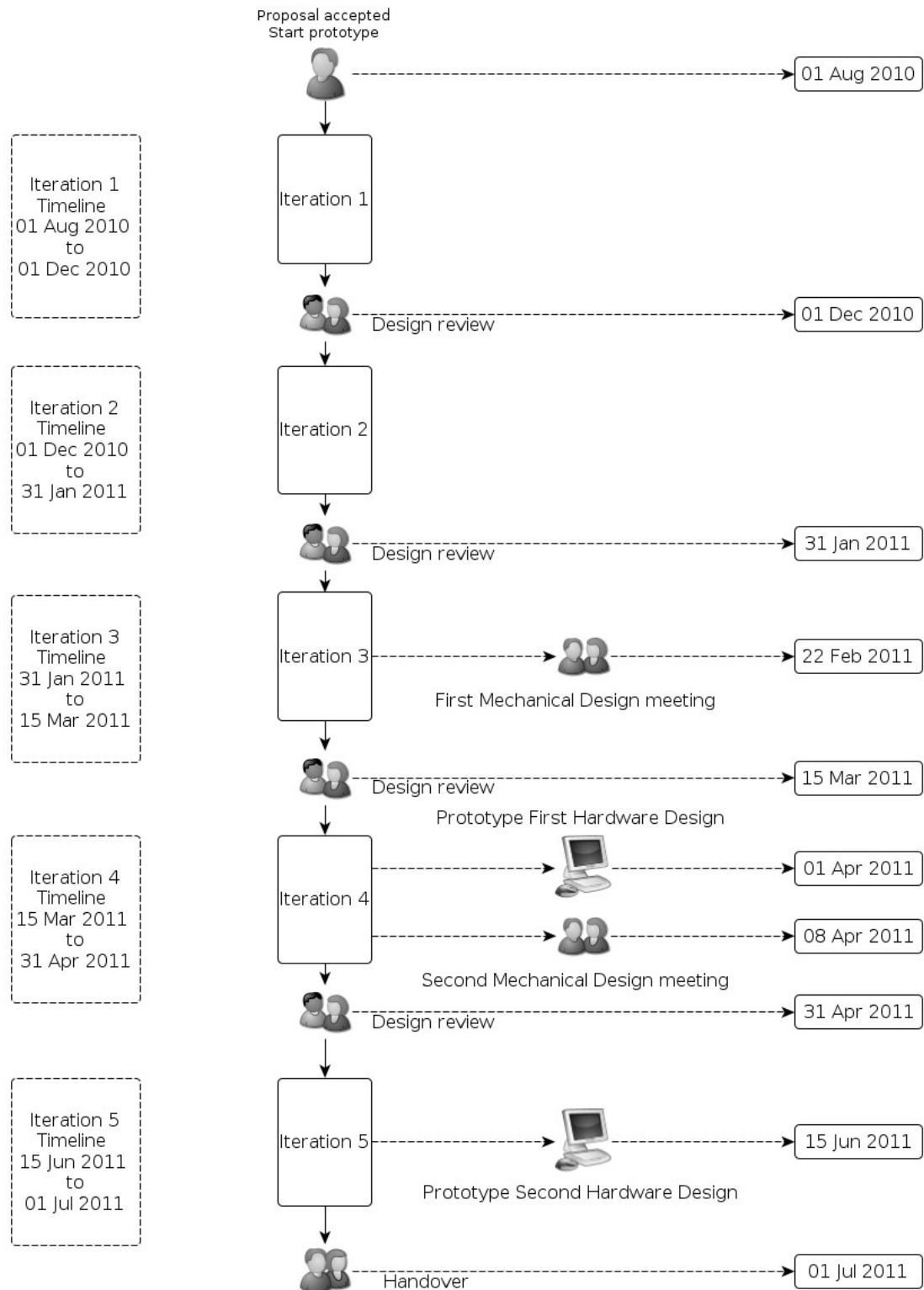


Illustration 1: Project Plan

1.6.1 Timeline

The project will use an iterative design approach, and as such the timeline is not completely set in stone at the start, but the following milestones are used as a guide:

- **milestone:** Project Started (Proposal In) **01 Aug 2010**

First Iteration (approximately 3 months)

- Test jig prototype schematic capture complete +-3 Weeks
- Test jig prototype PC board layout done +-2 Weeks
- Initial application design (design Interfaces) +-2 Weeks
- Prototype test jig PC board and fix any problems +-1 Week
- Documentation +-1 Week
- **milestone:** Design Review **01 Dec 2010**

Second Iteration (approximately 8 weeks including December holidays)

- Use first iteration design review to change hardware schematics and PCBs, add first test jig stack +-2 Weeks
- Expand application to include standard tests +-2 Weeks
- Documentation +-1 Week
- DECEMBER HOLIDAY +-2 Weeks
- **milestone:** Design Review **31 Jan 2011**

Third Iteration (approximately 6 weeks)

- Use second iteration design review to change hardware schematics and PCBs, update test jig stack +-2 Week
- Expand application to include EOS standard tests and add signal injection +-3 Weeks
- Documentation +-1 Week
- **milestone:** Meet with Mechanical Engineering dept for first mechanical iteration design meeting **22 Feb 2011**
- **milestone:** Design Review **15 Mar 2011**

Fourth Iteration (approximately 6 weeks)

- Use third iteration design review to change hardware schematics and PCBs, update test jig stack +-2 Week
- Expand application to include programming and label printing functionality +-2 Weeks
- **milestone:** Prototype hardware with first mechanical prototype **01 April 2011**
- **milestone:** Meet with Mechanical Engineering dept for second iteration design meeting after prototyping **08 April 2011**
- **milestone:** Design Review **31 April 2011**

Fifth Iteration (approximately 8 weeks)

- Use fourth iteration design review to change hardware schematics and PCBs, update test jig stack +-2 Week
- Expand application to auto detect communications ports and add network printing +- 2 Weeks
- Documentation +- 1 Week
- Write operators manual, installation manual etc. +- 2 Weeks
- **milestone:** test full system using second iteration hardware design and final design of generic test jig and test jig stacks **15 June 2011**
- **milestone:** handover to production **01 July 2011**

1.6.2 Budget

The budget for the prototype system of the generic test jig will be R40 000. This value does not take into account the salary of the author or any of the people he interacts with. It does include the following:

- Components worth R5000 (mostly from the Util Labs in house store, see individual BOMs for more information about individual components) R5 000
- The first set of jig mechanics R20 000
- The cost of the PC used on the test jig prototype R6 000
- The cost of manufacturing the PC boards depending on boards

Once production ready jigs can be created, a manufacturing cost can be calculated for each individual jig type created in the framework.

1.7 CONCLUSION

Testing of product is a such a core function in the manufacturing process that is currently causing a large amount of time and money to be lost on design time and downtime due to the current set of test jigs. Expansion of the Util Labs product portfolio is ongoing and as such it is necessary to increase the testing capabilities of the company.

Once implemented, this project should create not only allow for the creation of reliable test jigs, but jigs that can in the case of failure be replaced or repaired in no more than half an hour to minimize cost and loss of production time. The created test jigs should also increase the manufacturing capabilities of Util Labs by supporting the scalability of production, as well as creating tools that will allow for greater quality to be built into the product. The design time of new test equipment should be kept to a minimum by using a solid base of reliable design sub-components.

CHAPTER 2

2.1 INTRODUCTION

In this chapter different testing systems are evaluated and an idea is created of the solutions available in the current testing market. Using the information gained in the literature survey, an initial design is proposed to conform to the requirements set forth in the user requirements specification. The electrical and functional requirements for the design are stated at the end of the chapter.

2.2 LITERATURE SURVEY

The concept of bed of nails testers is not new and as such there is a wealth of information available on their design and implementation. A lot of the systems referenced however do suffer from the problems outlined above, although to differing degrees. An elegant alternative solution put forth is to use a fixture-less in circuit test solution called a flying probe tester. An example of a flying probe tester can be seen from Seica Technologies_[3]:

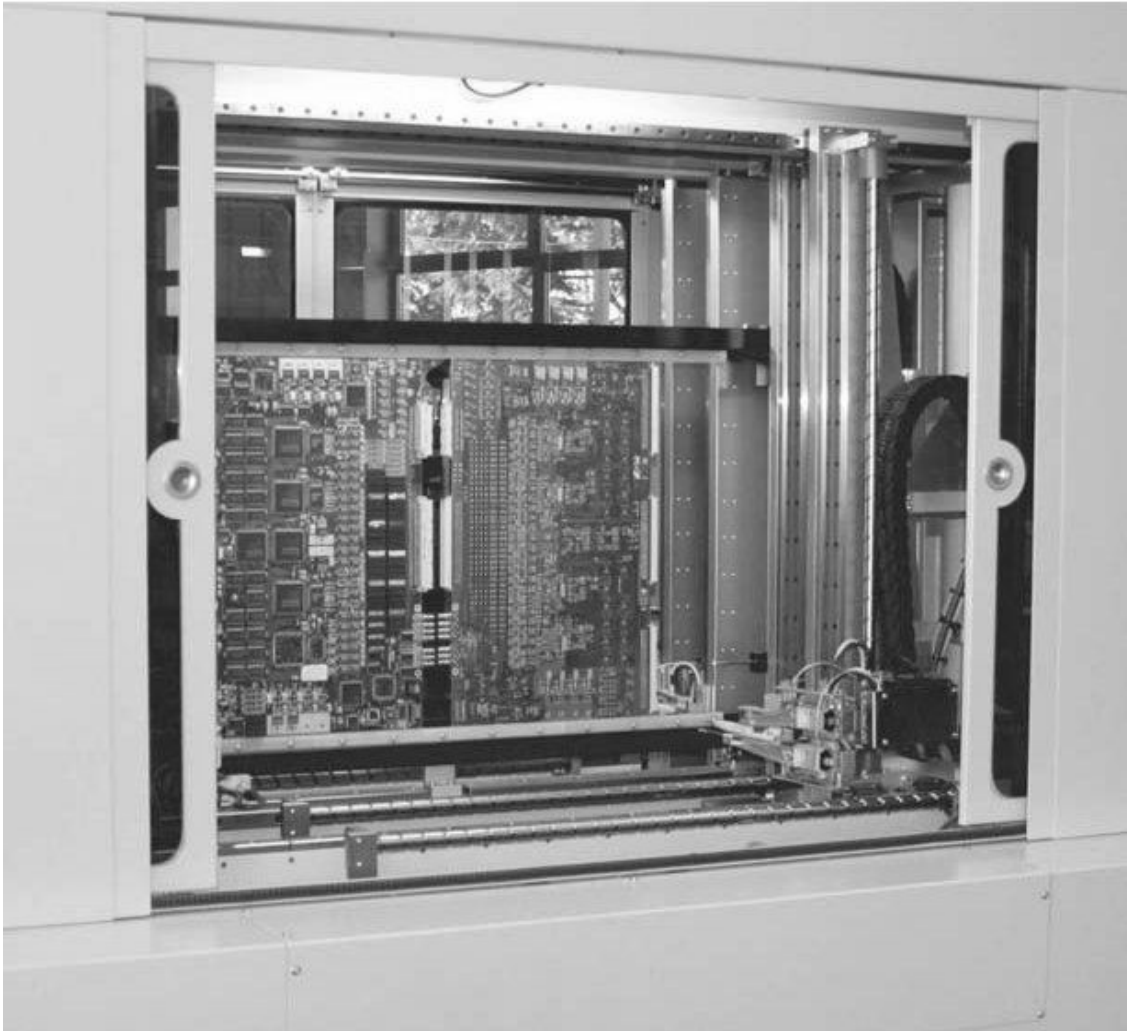


Illustration 2: Floating Probe System

(image retrieved from _[3])

In this system instead of having a bed of nails physically, a virtual bed of nails is created with a number of points that need to be tested and this virtual bed of nails is programmed into the system. During the test, the test probes of the machine are physically moved to each of the x-y coordinates of the test points in turn, injecting the appropriate signals, and testing each point to a predefined pass criteria before moving on to the next. The

advantage of this system and other types of fixture-less in circuit test solutions is that they address the flexibility requirement put forth in the problem statement as well as having a very short development turn around time ^[4], however the down side of this system is that the initial costs are exorbitant and the system cannot scale at all due to very slow testing times per board. These kinds of system are generally used to test prototypes, or in some cases to test bare circuit boards on panels.

A second alternative option is also available in the form of boundary scan technology. This technique of testing is best known through its JTAG implementation by the Joint Test Action Group (JTAG) which is widely used in debugging and programming of microprocessors and programmable logic chips ^[5]. In a test system that uses boundary scan technology, each of the input and output pins of the chips connected on the JTAG scan chain are scanned via a set of registers and their respective levels compared to known good values during various phases of the test. The disadvantage is that the PC board being tested using this technique needs to be designed with this technique in mind, and it is very difficult to implement on to an older design if JTAG testing was not taken into account from the very beginning. Another disadvantage is that using the scan chain, only those levels that are visible through processor I/O pins are measurable, and some parts of the system cannot be tested. The advantage to this system though is a very low test jig implementation cost and a very high level of test integration with the design. Although I do not wish to follow the route prescribed by JTAG testing, I do feel a lot can be learned from the implementation and in the initial design I will be using a JTAG programmer to program the microprocessors of the units under test on the system.



Illustration 3: JTAG programmer

(image retrieved from ^[6])

The third and final alternative option is to have designs with built in self test . Systems that have this as part of the design have the highest level of system integration during the design phase, as during the design process each unique test needs to be added to the system as another “component”. It minimizes the testing time, as well as the need for complex test hardware, but greatly increases design time and product cost. This sort of system however is widely used in military and medical systems where a very high level of testing, as well as continual testing is required. It is however not really suited to more commercial products as the additional expenditure can normally not be justified [7].

The following table shows a summary of some of the advantages/disadvantages of the different kinds of testing solutions:

<i>Solution</i>	<i>Advantages</i>	<i>Disadvantages</i>
<i>Bed of nails</i>	Simple operation, handles very high volume, very rugged, very wide test coverage	High initial cost, little scalability, cannot easily be modified, can damage the PC board with sharp pins
<i>Fixture-less testing</i>	Very flexible (once initial cost is paid subsequent changes are negligible), easy to retrofit, very wide test coverage.	More complex operation, cannot handle high volume, slow test speed, very high initial cost
<i>Boundary scan</i>	Low initial hardware cost, high system integration	High design cost (must be designed into system), very difficult to retrofit, not 100% test coverage
<i>Built-in test</i>	Very high system integration, low hardware cost	Very high design costs, must be designed into all parts of the system, Cannot be retrofitted

Table 1: Manufacturing test solutions

The system I aim to develop in this project takes from all of the above solutions. I aim to combine the ruggedness of the bed of nails and add some of the flexibility of fixture-less testing and combine this with boundary scanning technology for programming and allow for a high degree of self test in designs. By taking the strongest points of each of these approaches I feel that a flexible, scalable and affordable test jig system can be created.

2.3 PROPOSED PRACTICAL DESIGN

The basic system outline is shown in the diagram below:

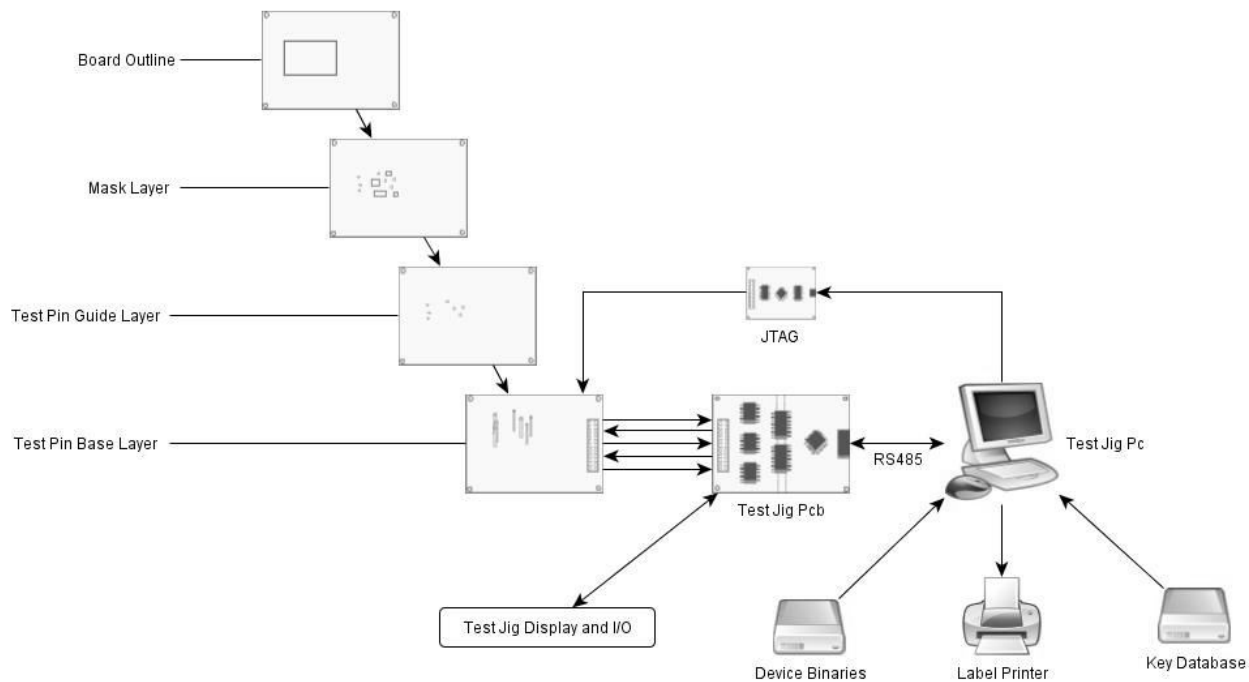


Illustration 4: Proposed Test Jig System

As can be seen in the above image, in the proposed system a standard PC will be used to interface to a generic test jig PCB, which will house all of the measurement hardware, which will in turn interface to a bed of nails that contains the signal conditioning hardware, and is also the main interface to the unit under test. The PC will also interface to the programming hardware, as well as some standard peripherals. The individual components are described in more detail in the pages to follow.

2.3.1 Test Jig PCB

The test jig itself will be based around an ARM7 Cortex processor running a variant of the EOS embedded messaging operating system. The Jig will also need to conform to the following specifications (these however may change to add new requirements as needed):

- 10 binary inputs
- 10 voltage inputs
- 4 frequency inputs
- 2 counter inputs

- 2 serial communication lines to unit under test
- 10 binary outputs (for signal injection)
- Jig information interface for when jigs are running in parallel
- keypad or buttons to operator the jig
- RS-485 or equivalent for communication with PC
- Permanent storage for test results and test scripts (flash memory or SD card)
- Permanent storage must be accessible via Windows/Ubuntu (FAT file system)

These requirements may however at this point not reflect a complete overview of what is required, and certain I/Os may need to be added to ensure full test coverage. If insufficient I/O is available, then an additional processor may be added with one processor acting as a master, and the second acting purely as a slave I/O processor.

The I/O to the unit under test needs to be electrically isolated from the rest of the system to ensure that during fault conditions the rest of the system is protected. In addition to being electrically isolated, all of the ICs that are electrically connected to the unit under test need to be socketed through-hole ICs so that in the event of a failure, they are easy to swap out very quickly.

The permanent storage will need to hold the test jig script that determines what will be tested on which I/O lines to the unit under test. This script needs to be transferable easily to a second jig in the case of failure to speed up the hot-swap process on the manufacturing line. The scripting language used will be either an open source standard scripting language, or a custom simple scripting language that is specific to the project, depending on whether a scripting language is found that has all of the features required and has a small enough footprint to be placed on the ARM processor. An alternative solution can be to create the tests to be run as a component in the test jig firmware – although this will need the test jig code to be recompiled for each test and the jig itself will need to be reprogrammed, this will allow for a MUCH faster swap out time, as an identical test jig(s) can be kept on stand-by in case of failure. Both options should be prototyped.

2.3.2 Test Jig PC

The test jig PC will run a Java application with a Swing GUI implementation that shows the current status of the Jigs(s). The PC will provide storage to serialized binary files for the unit under test and communication to a central production server. By splitting the

programming section away from the test jig itself, different kinds of programming can be supported by the system and a simple interface can be maintained between the test jig and the PC. This same interface will also allow access to normal PC peripherals like label printers.

2.3.4 Bed of Nails

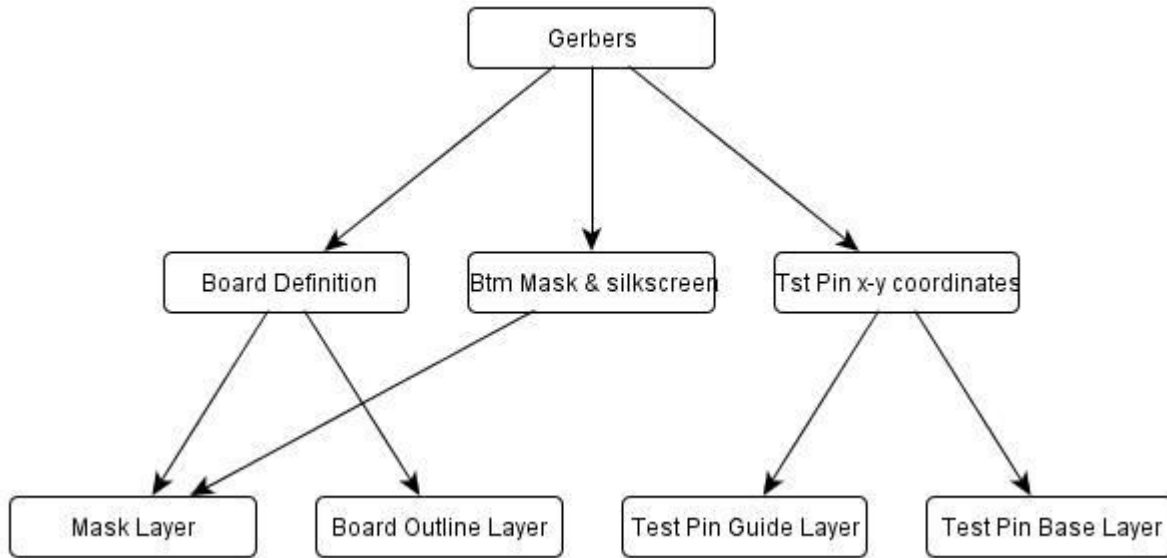


Illustration 5: Bed of Nails layers

The bed of nails forms the interface between the unit under test and the test jig system, and in many ways is the core component that will solve (or attempt to solve) many of the problems put forward in the problem statement. The bed of nails consists of the following 4 components:

- Test Pin Base Layer
- Test Pin Guide Layer
- Mask Layer
- Board Outline Layer

Each of the layers is generated off of the gerber files for the PC board that is to be tested. The functions of each layer and how they are generated are as follows:

- **Board Outline Layer**

The board outline layer acts as a physical guide to align the unit under test with the test pins. It is generated from the board definition layer of the gerbers by drawing an outline around the board and using it to make a cut out on the board outline layer PCB.

- **Mask Layer**

The mask layer guides the test pins to the right test pads on the unit under test, and acts as a base for the unit under test to rest on. It is generated off of the bottom mask layer and silkscreen gerbers that define where there are component leads and physical components. Cut outs need to be made on this layer for components on the bottom layer, and holes need to be drilled for test pins to go through, but also for through hole component leads that are fitted on top of the board.

- **Test Pin Guide**

The test pin guide gives stability to the spring loaded test pins and forces them to stand up straight and align with the unit under test correctly. The board is generated directly from the x-y coordinates of the test pins so a hole is made for each one of them.

- **Test Pin Base**

The test pin base is where all of the test pins are soldered down and are routed to a standard connector that interfaces with the test jig PC board. The signal conditioning for the signals to and from the unit under test is also on this board. The board is generated from the x-y coordinates of the test pins. The test jig script will identify which test pins go to which pins on the connector and what voltages etc need to be measured there. Different Test Pins will be standardized on depending on what interface is available to the unit under test. The pins generally used can be seen in the following image:

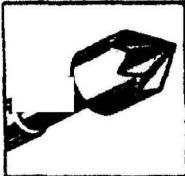
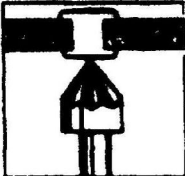
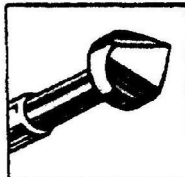
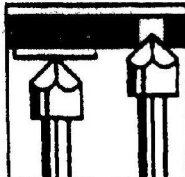
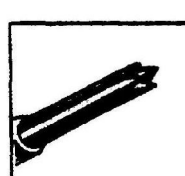
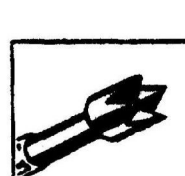
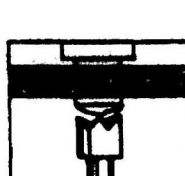
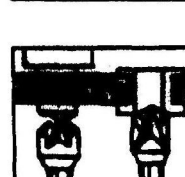
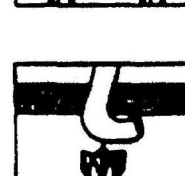
		<u>Name</u>	<u>Application</u>
		Star	Plated-through holes, lands, pads; self-cleaning
		3- or 4-sided Chisel	Lands, pads, holes; self-cleaning
		0.040 Crown	Lands, pads, holes; self-cleaning
		0.050 Crown	Leads, pads, lands; self-cleaning
		Tapered Crown	Leads, pads, lands, holes; self-cleaning
		Tulip	Long leads, terminals, wire-wound parts; self-cleaning

Illustration 6: Different types of test pins

Image retrieved from [8, page 74]

2.4 CONSTRAINTS

In order to ensure that the framework performs as required by the URS, the following 3 constraints are put in place to ensure that the process of creating a new test jig from

existing design files works as smoothly as possible:

- **All the test points must be available from 1 side of the unit under test**

This will ensure that the simplest test pin interface can be used, as only 1 axis of movement is required to bring the test pins into contact with the unit under test. If there are test points that need to be measured from the other side, additional design time will be required.

- **The communication interface must run on the standard message bus**

This ensures the maximum amount of re-use of the communications interface, and allows for fast turn around times in test jigs. If however a second message catalog needs to be implemented with a network bridge in between, it will require additional design time.

- **Movement of test points between versions should be minimized.**

If the test points stay stationary between versions, then the same beds of nails can be used for the different versions. If however any test points move, all the boards in the bed of nails stack need to be changed, and this requires additional time in designing and prototyping.

Appendix A is a guideline to placing test points on future designs to ensure that they are easy to use.

2.5 **TEMPLATING**

In order to facilitate the fast development of hardware and software in the framework, the parts of the design that are common between all bed of nails implementations need to be split off into hardware and software templates. The main templates that need to be created are:

- **The bed of nails stack boards**

Each of the boards in the bed of nails stack (outline, mask, guide, base) always perform the same tasks on all of the bed of nails implementations. As such they can all have a standardized outline as well as standardized input and output connections (for those ones that have input and output connections), and just have the positions of the test points changed between different test jig implementations.

- **Generic board interfaces**

Each of the bed of nails boards perform the same tasks, so many of the functions can be grouped together in software into a template and just the hardware mappings changed for individual boards. This templating forms the basis of the `GenericTestBoard` class in the software framework, where each descendant class will just add the hardware mapping.

- **Test Jig implementations**

Each of the test jig implementations will obviously run different tests since they are each for different products, but many of the values that are tested are done so in standard ways (voltage measurement, state measurement etc) and these should be grouped together intelligently so that only the reference values and other essential parameters need to be specified when a test sequence is set up. These test functions will form part of the `GenericTestJig` class, where each descendant class just adds the specific tests needed to run and the sequence.

2.6 GENERAL USE CASE

The main person to interact with the test jig will be the operator, and as such, his interaction is the major use case against which the test jig will be tested. The test process will loosely follow the structure shown in the diagram below:

- **Insert PCB (operator action)**
 - Before the operator inserts the unit under test, the LCD panel must indicate to him/her that the jig is ready to accept a unit to test, or else show some error code that indicates it is not ready.
 - If it is ready the unit under test is inserted into the test jig, and is physically clamped down using a standard physical clamp.
- **Start Test (operator action)**
 - Once the unit under test is inserted, the test is started by pressing an appropriate button on the interface.
- **Run Pre-tests**
 - A number of tests are sequentially run against the UUT, (testing voltages of power supplies, checking oscillators etc), and once all the pre tests are run, the test jig sends a message to the test jig PC that it is time to program the unit under test.
- **Program**
 - The UUT is programmed by the test jig PC using the JTAG interface.
- **Run Post-tests**
 - Once the unit is programmed through the PC, a second set of tests are run that checks all of the functionality of the unit (can data be written and read to the memory, can serial data be sent and received etc.) and once these tests are passed, a result is displayed to the operator
- **Result**
 - A station pass label is printed for the unit. If at any point in the process the unit fails, then the operator is informed, a label indicating what test has failed is printed so that the unit can be fed back into the system for rework.
- **Store Result**
 - A log is stored of the results attained.

- **Remove UUT (operator action)**
 - The tested board is removed from the test jig and sorted according to the printed label.

The test flow only requires the operator to perform 3 actions nl.:

- Load the board to be tested
- Start the test
- Remove board and sort

If there are additional actions required of the operator in an implementation of a test jig, then they should be of a similar nature.

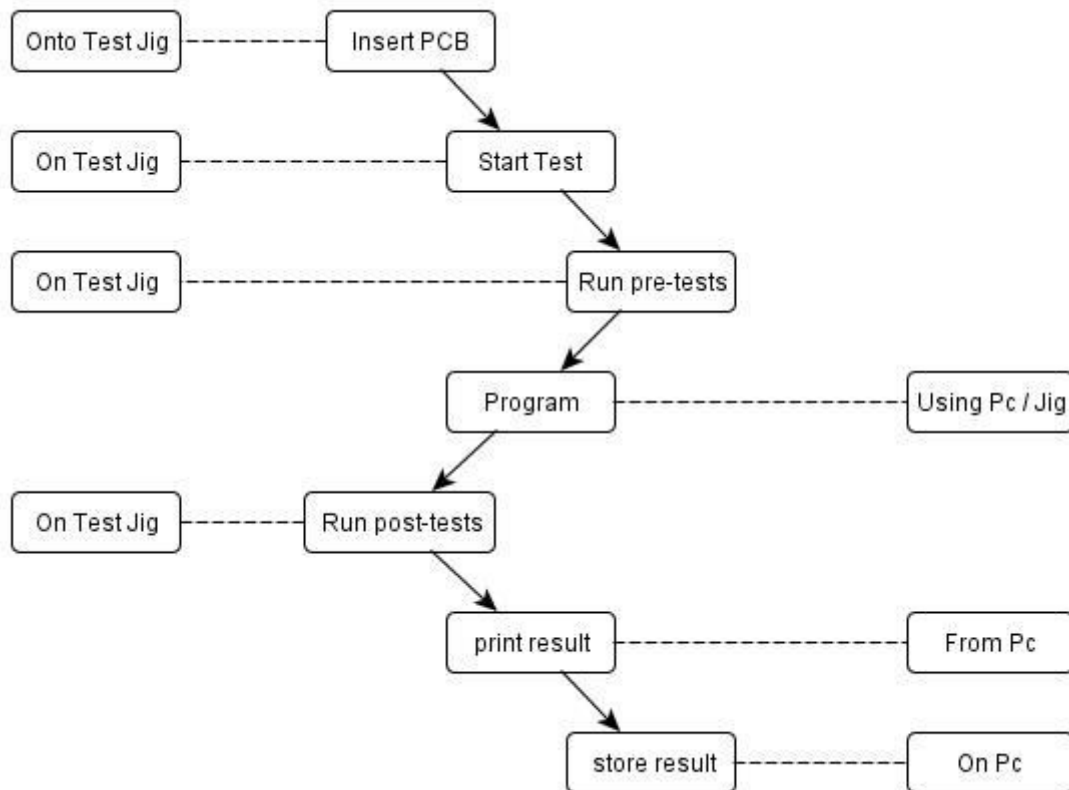


Illustration 7: The standard test flow

2.7 PRODUCT SPECIFICATION

Using the requirements and restraints placed on the design above, below is a preliminary specification of the system:

Test Jig Functional Specification:

- The test jig must be able to measure at least 10 binary inputs.
 - Input should be able to tolerate voltages up to 20V
 - Input impedance of $> 100k\Omega$
- The test jig must be able to measure are least 10 voltage inputs.
 - 1% or better accuracy
 - Input should be able to tolerate voltages up to 20V
 - Input impedance of $> 100k\Omega$
- The test jig must be able to measure are least 4 frequency inputs.
 - Must be able to measure square wave frequency up to 2kHz
 - Input should be able to tolerate voltages up to 20V
 - Input impedance of $> 100k\Omega$
- The test jig must be able to measure are least 2 digital counter inputs.
 - Must be able to count signals of up to 2000 pulses per second
 - Input should be able to tolerate voltages up to 20V
 - Input impedance of $> 100k\Omega$
- The test jig must have at least 2 serial communication lines to the unit under test.
 - Standard UART communication is 115200 baud, 8bits, no parity, 1 stop bit
- The test jig must be able to inject 5 or more signals.
 - Signals up to 1kHz must be able to be injected
 - Injection lines must be able to handle 230Vac
 - Injection lines must be able to handle up to 1A continuous

Electrical Specifications:

- The test jig will be supplied with 230Vac mains voltage to power it, and must require no external power supply.
- Power to the test jig will be supplied via a standard IEC power connector.
- The test jig, when not injecting signals must draw less than 2W of power

- Isolation between the UUT and all external communications lines must be $> 1\text{kV}$
- Input impedance on all inputs must be $> 100\text{k}\Omega$
- All inputs must be 20V tolerant
- The test jig must be fused
- No Voltages should be present anywhere a user can physically access during testing

Interface Specifications:

- Test result storage must be non-volatile and be able to store at least 10 000 results.
- Test results need to be accessible to a windows or Linux operating system
- Once Installed, the required user interaction should consist of no more than single actions, for example: close lid, push button, scan label etc
- RS232/RS485 communication will take place at 115200 baud, 8 data bits, no parity bit, 1 stop bit
- Graphical communication to the user must all be in English

2.8 CONCLUSIONS

Using the literature survey and the user requirements specification set up earlier, a proposal was put forward for a design framework that should address the problems as set forth in the problem statement. The design will use a standard PC to run an application that controls a standard test jig board that is generic for all of the test jigs created within the framework, which in turn interfaces to a stack of 4 boards that create the bed of nails. This assembly will fit inside of a standard enclosure which will allow for the test jigs to be hot-swappable as well as giving them a standard interface that an operator can use to initiate the tests. Using the iterative approach, this design will be built up in stages and should, if implemented correctly address all of the needs of the relevant stakeholders.

CHAPTER 3

3.1 INTRODUCTION

In this project, the design methodology followed will be an iterative design process, which will include a substantial testing component during the development cycle, as well as a testing phase at the end of development to ensure that the best solution is come to.

The design is to be implemented with a specific guinea pig board in mind, so as to test all parts of the process. The end result should be the generic test jig framework, as well as the jig for the guinea pig board. A second jig will also need to be implemented to ensure that the “speed” requirements of the project have been fulfilled.

As the design process was followed through the individual iterations, at the end of each iteration, the interested parties were consulted (as they were available) to ensure that the project was still on track for what they initially wanted out of it in the design meetings. As time progressed, a number of changes were made to the way in which the test jig is designed, manufactured, used and tested, based in the input from the interested parties. In a product orientated environment, this design methodology that continually takes feedback from the stakeholders is the best methodology to follow, as it doesn't necessarily deliver a product that matches exactly the initial design specification, but it does deliver the product that the stakeholders need and that is ultimately what is important.

In documenting the individual iterations, only those parts that are specific only to that iteration will be mentioned in this document to ensure there is no repetition. The final iteration will contain all of the design information for the system in its completed form.

3.2 THE PROTOTYPE

The initial prototype was based on the design concept outlined in chapter 2.

3.2.1 Prototype Characteristics

Board	Rev
Generic Base (Proto)	V1
MCU bed (Proto)	V1/V2

Table 2: Hardware designs associated with the prototype

The prototype is characterised by the following main points:

- RS485 communication between PC and test jig
- operator interfaces to the jig using buttons
- LCD screen to give feedback to the operator
- test scripts are stored on the test jig on an SD card
- test results are stored on the test jig on an SD card
- a number of isolated ports and isolated power supplies
- isolated analog measurement – isolated port to an external chip
- fixed interface to a standard test jig bed of nails base.
- Debug communication is handled by the test jig
- configurable I/O ports (input/output)
- 2 processors with a inter-processor bus (explain how exposing the message bus works to create an inter processor bus.)

The main work done was to create the first hardware platform, as well as the first bed of nails stack. The initial port of the EOS operating system to the board was done, and All of the firmware to support the RS 485 communication , inter processor bus and basic I/O was implemented on the prototype. For additional information on the RS 485 and interprocessor bus firmware see the following files in Appendix B.2:

- `drv_Rs485.h` and `drv_Rs485.cpp`

- brg_Rs485.h and brg_Rs485.cpp
- brg_processor.h and brg_processor.cpp
- brg_processorMaster.h and brg_processorMaster.cpp
- brg_processorSlave.h and brg_processorSlave.cpp

This iteration also allowed for the testing of the initial design process to create a set of test jigs and the code to support it.

3.2.2 Detailed Design Description

For the full schematics and gerbers of the Generic Base prototype board V1, see Appendix B.1.

The design of the generic base board can be broken up into the following parts:

- Power supplies
- Controllers
- Isolated bi-directional ports
- Analog
- UUT comms
- Storage
- LCD

Each of the individual sections are described below.

3.2.2.1 Power Supplies

The power supplies on the generic test jig board are made up of 4 isolated sets of +3.3V and 5V power supplies. The sets are isolated so that difference Analog front ends and bi-directional ports can interface to UUT that have different floating ground potentials. Three of the isolated power supplies are used to connect to the UUT, and the fourth is used for the on board controllers and communication. Using the FEA (Front End A) supply as an example in the illustration below:

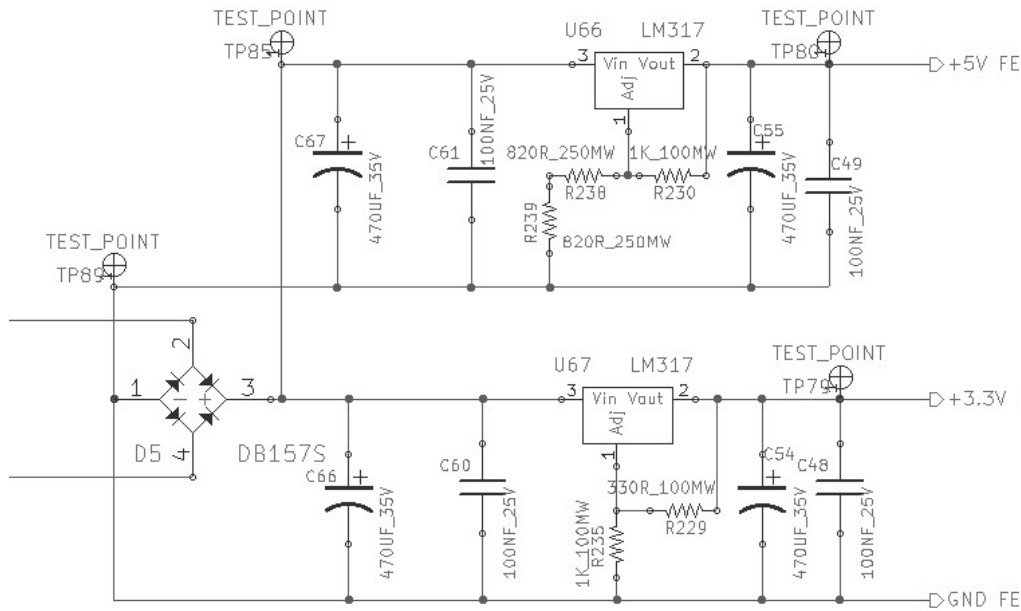


Illustration 8: Isolated +3.3V and +5V power supply set

Each isolated power supply set is created using an isolated secondary 12Vrms winding from one of the two Zettler BV302D12028 transformers (L1 and L2), and then feeding the voltage into a DB157S/B380S bridge rectifier (D5) to get a ripple DC voltage out with an approximate peak of 16.8V. The ripple DC is then smoothed out using two 470uF 35V electrolytic capacitors (C66 and C67) and fed into two separate LM317 (U66 and U67) linear regulators with two 0.1uF decoupling capacitors placed close to the regulator input pins (C60 and C61). The output voltage from the LM317 is determined by:

$$V_{out} = V_{ref} \left(1 + \frac{R_2}{R_1} \right) + I_{adj} R_2$$

Where:

V_{out} = the output voltage

V_{ref} = 1.25V reference voltage

R_2 = ground resistor value

R_1 = output resistor value

I_{adj} = 100uA leakage current

In the above power supply for +3.3V FEA:

- $R_1 = R_{230} = 1k\Omega$ and
- $R_2 = R_{238} + R_{239} = 820 + 820 = 1.64k\Omega$
- calculated V_{out} of 3.3V

and for +5V FEA

- $R_1 = R_{229} = 330\Omega$

- $R2 = R235 = 1k\Omega$
- calculated V_{out} of 5.04V

Output ripple is controlled with two more 0.1uF decoupling capacitors (C48 and C49) and two more 470uF 35V reservoir capacitors (C55 and C54).

Test points are placed to measure +5V output, +3.3V output, Ripple DC input and GND using test points TP79, TP80, TP85 and TP89 respectively.

3.2.2.2 Controllers

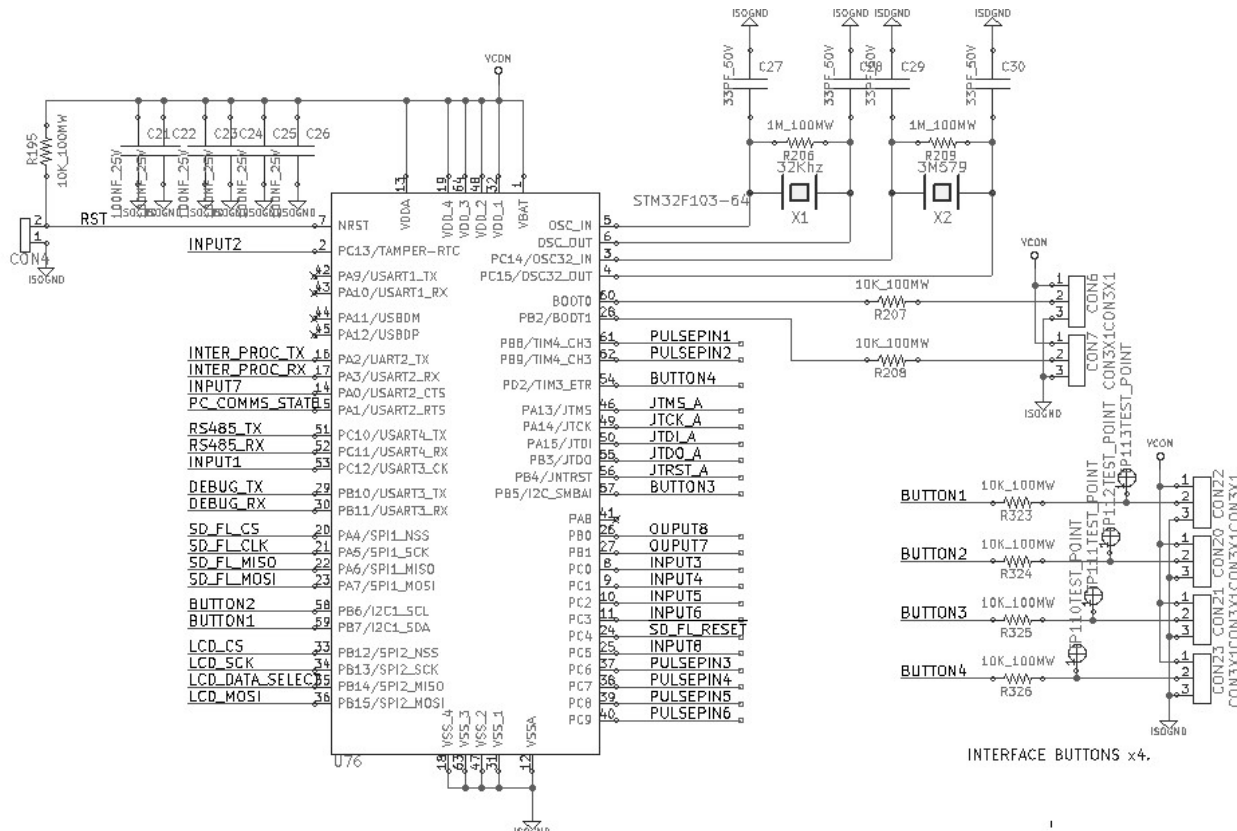


Illustration 9: Master Processor schematic

The controller chips are two STM32F103 64 pin ARM 7 cortex processors. These two processors control all of the generic test jig functionality. The two processors are designated master (U76) and slave (U75). The Master processor is responsible for RS485 communications with the Host PC using the RS485 driver chip (U58), interfacing to the user through the 4 SPDT switch connectors (CON20, CON21, CON22 and CON23), writing to the LCD through the SPI2 port, writing to and reading from the SD card and measuring signals on pulsed inputs. The Slave processor is responsible for measuring analog voltages using the isolated SPI port to the AD7918 ADC chips (see section 3.2.2.4). The processors share the responsibility of measuring the isolated inputs and outputs.

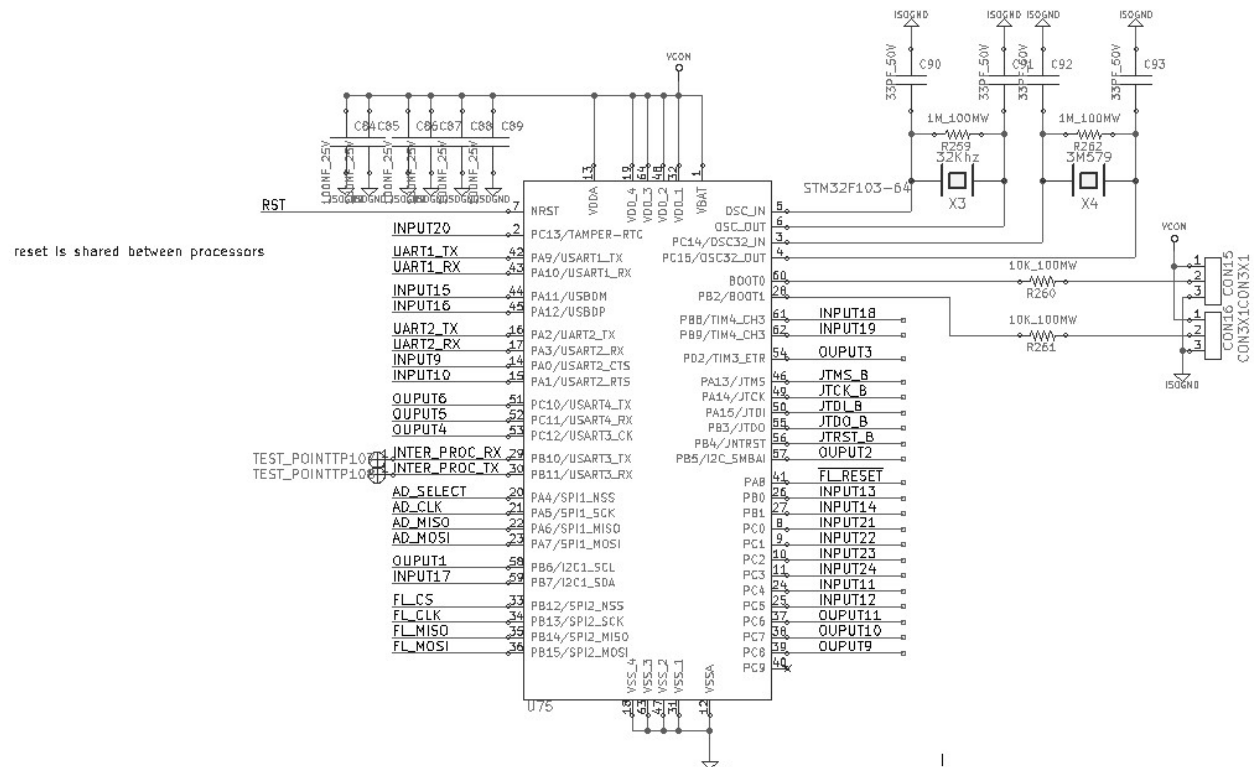


Illustration 10: Slave Processor Schematic

Each of the processors have their own 32 kHz watch crystals (X1 and X3) and their own 8MHz clock crystals (X2 and X4). Programming (and debugging) of both of the processors is done via JTAG interfaces on P1 and P6:

In this set up however the pull up and pull down resistors are optional, as the STM32 processor has internal pull-ups and pull-downs on the JTAG port [10, page 1050 section 31.4.3]

The boot state of the processors can be set individually using jumpers (CON6,CON7 and CON15,CON16). To boot from internal flash on both processors these jumpers all need to be set to GND (shorting pin 2 and 3). If the BOOT0 pin is set low (CON6 and CON15) then the state of BOOT1 is irrelevant and the processor will boot from internal flash [9,page 6 table 1], but the jumper is placed on BOOT1 to allow provision for times when the processor should not be booted from the internal flash.

The power supply rails to the processors are decoupled using 0.1uF capacitors (C21-C26 and C84-C89).

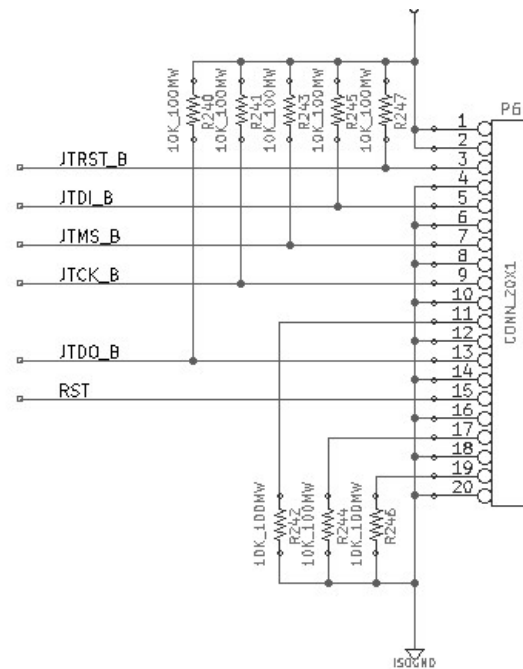


Illustration 11: Jtag Interface

The two processors act as a single unit, since their internal message busses are virtually tied together using the inter processor bus (Signals INTER_PROC_RX and INTER_PROC_TX on test points TP107 and TP108 respectively). To understand how this works some understanding of the nature of the EOS message bus is required (information on the EOS operating system is available in Appendix G).

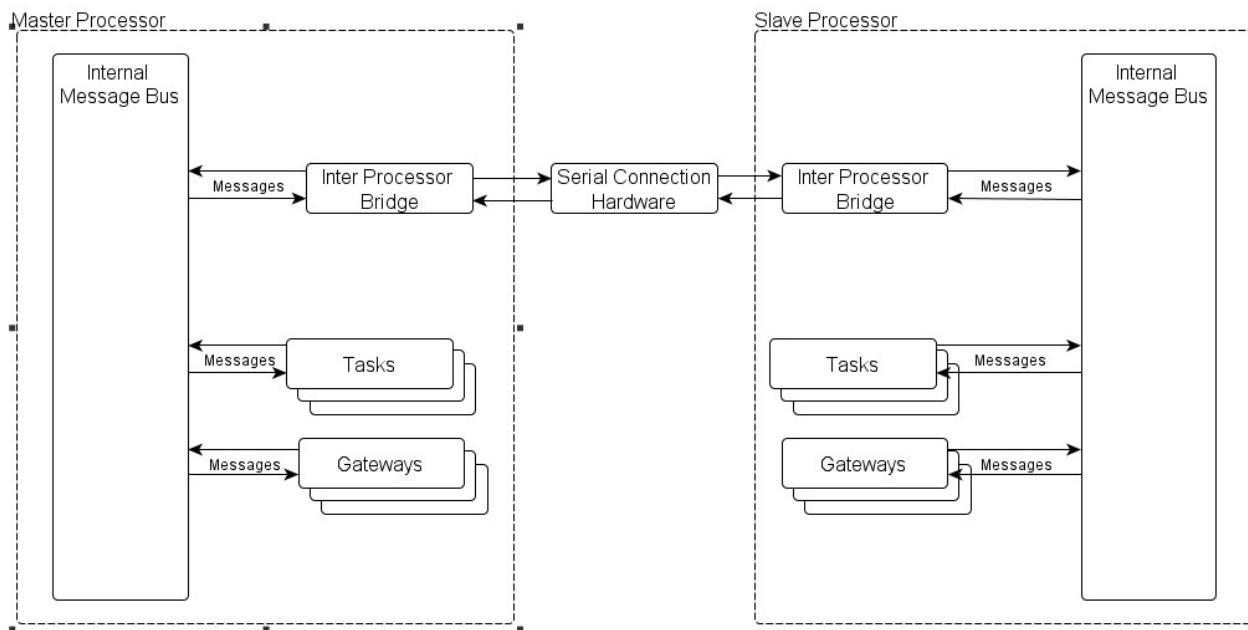


Illustration 12: Inter Processor Bus

In this network bridge implementation, the internal message busses of the two processors are tied together via a UART on each processor controlled by an Inter Processor bridge that just reads messages on the message bus it is connected to, filters them and sends them to the other processor. This means that messages sent by tasks and gateways on the slave processor can be seen by tasks and gateways on the master processor and visa versa (provided they do not get filtered by either bridge's filter method). This system allows the two processors to react as 1 processor sharing all of the processing tasks between them. The reason this system was implemented on the prototype was to allow for the sheer number of input and output pins required.

Communication on the prototype was to be done using a standard MAX485 RS485 driver chip to the host PC's RS485 network.

2.5kV using the CNY74-4 opto coupler. The inputs are buffered using a 74HC541/74LV541 non-inverting buffer on both the input to the opto coupler as well as the output. The input to the opto coupler is buffered so as to not put load on the UUT (the opto coupler is driven at 5mA, whereas the 74LV541 has a leakage current of 1uA [18, page 4 table 6]) and the output of the opto coupler is once again buffered as the opto coupler collector is limited with a 10kΩ pull up resistor to compensate for any aging effects in the opto coupler (the minimum Current transfer ratio for the opto coupler is specified at minimum 50% [20, page 2, table CURRENT TRANSFER RATIO], but over time aging in the opto coupler can lower this value [14]). Using this system however, the input value is inverted through the opto coupler and this fact needs to be taken into account in the firmware. The same logic is applied in reverse to the output circuitry:

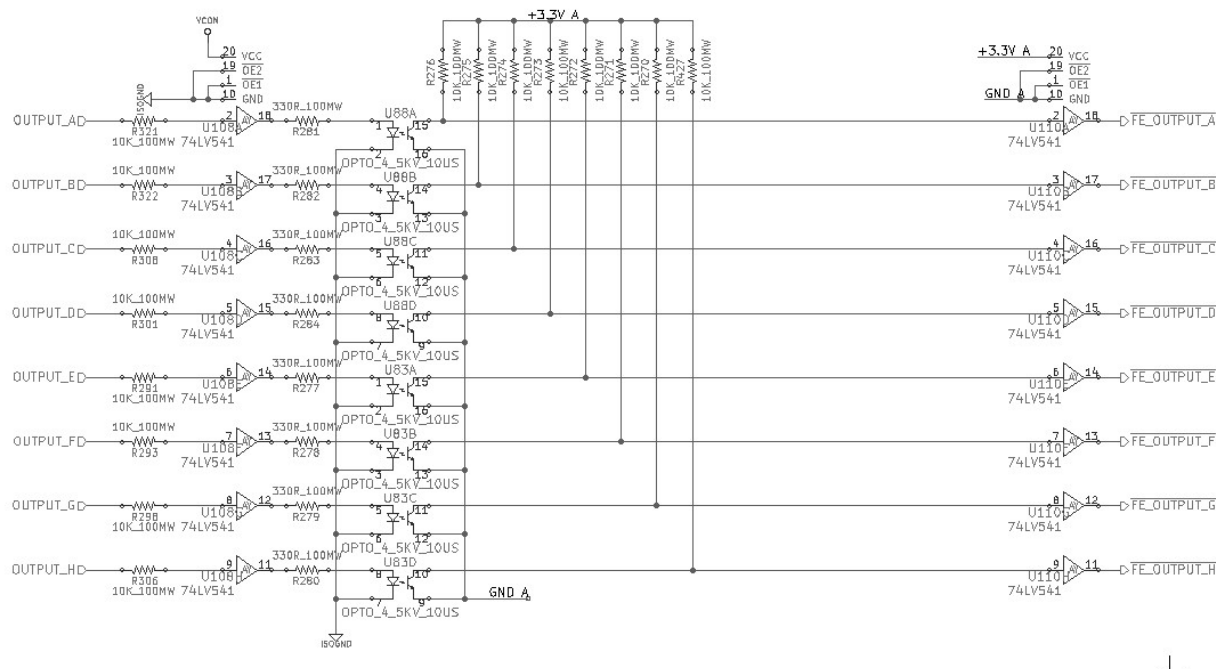


Illustration 16: Isolated Output Schematic

In the output circuitry, the exact same buffer chip is used to also drive the opto couplers (although the maximum current that can be sourced through the VDD pins of the STM32F103 chip is 150mA [19, page 41 table 8], with all of the outputs on the device driving opto couplers, this will cause excessive heating of the device without buffering) on the input side of the opto couplers, and then on the output side of the opto couplers the same buffering is used again to compensate for any aging effects. As with the input circuitry, the logical level of the output will be inversed through the isolation circuit, so this needs to be taken into account by the firmware.

The prototype board has 24 designated inputs to the test jig, 6 designated pulse measurement inputs and 11 outputs split across 3 ground potentials. These ground potentials can either be tied to the same point on the test jig stack if all of the signals on the UUT are referenced to the same ground potential, or they can be used individually as 3 separate circuits completely isolated from each other (and the test jig PC.)

3.2.2.4 Analog Measurement

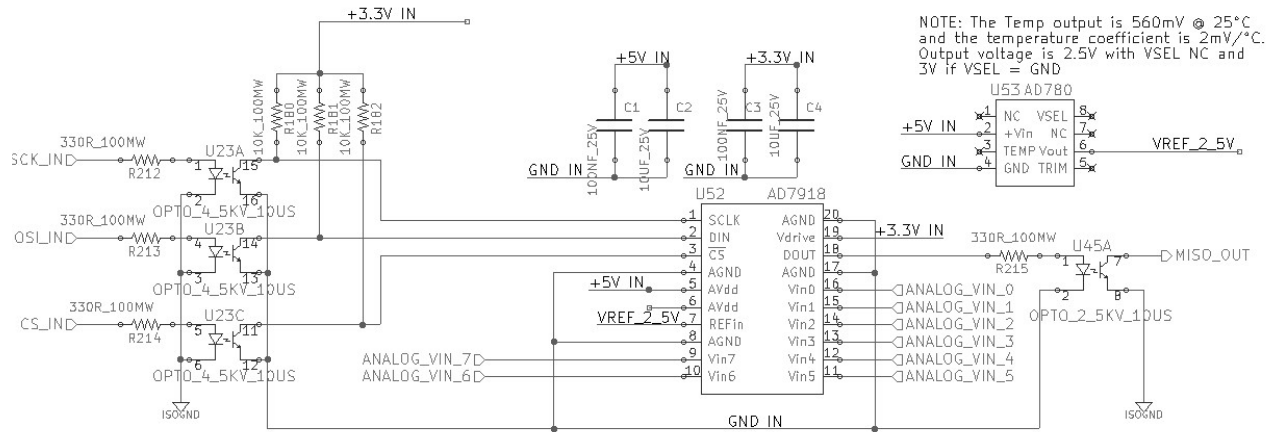


Illustration 17: Isolated Analog Front End

Analog voltage measurement on the test jig is achieved by floating an analog to digital converter chip and separate reference on one of the ground potentials to the UUT, and then isolating the SPI port communicating with that ADC chip. The chip in use is the AD7918 analog to digital converter from analog devices. The AD7918 offers 10 bits of resolution with a 0.5 bit error. Channel to Channel isolation is given as -85dB. The ADC converter has a low input impedance of 1uA and should not unnecessarily load the UUT [21, page 5, table 2]. The AD7918 will measure any voltage between 0V (the reference in this case that it is tied to on the UUT) and the supplied reference which in this case is fed from the AD780 reference chip. The output voltage used is 2.5V since the VSEL pin on the device is left not connected. The drift on the reference is given as 3ppm/°C [22, page 1]. A temperature measurement output is supplied on the reference chip via pin 3, but is unused since the ultra low drift is adequate for the accuracy required on the test jig.

On the prototype board a board modification was required to allow for communication to the chip as the opto couplers invert the signals on the SPI bus, so the opto coupler input tracks had to be cut and the pull up resistors changed to 0Ω links, and the SCK, MOSI and

3.2.2.5 UUT Comms



47

3.2.2.6 Storage

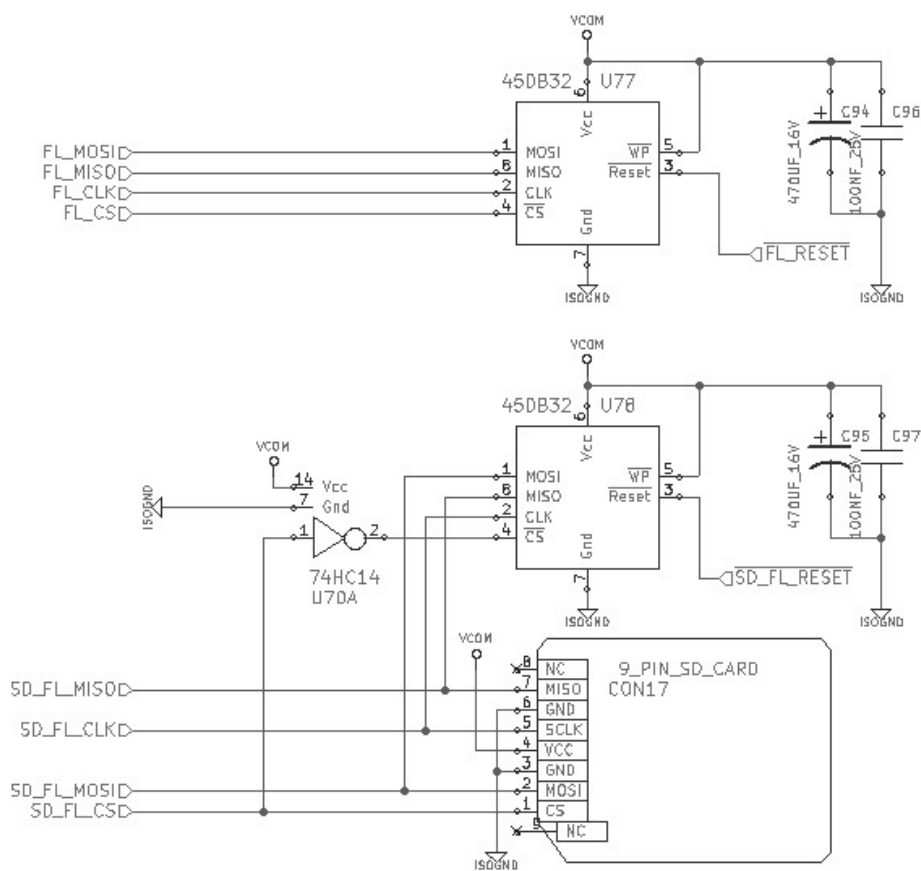


Illustration 19: Storage on SD card and Flash

Part of the requirements for the EOS operating system used is a Flash file system that is implemented on the AT45DB32 32Mb serial dataflash so settings can be stored. Provision was made for a standard SD card holder to allow for the settings, logs and test scripts for the test jig system.

3.2.3 The Test Jig Creation Process

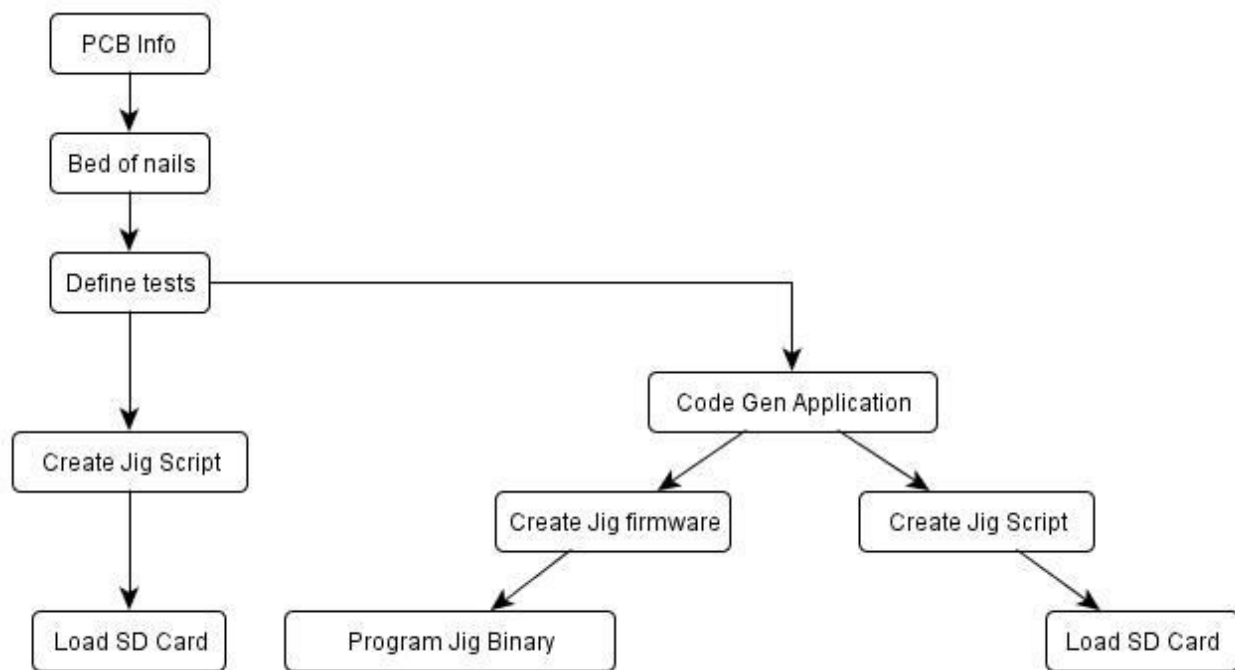


Illustration 20: Test Jig Design Process

The test jig creation process follows a number of predefined steps. On the prototype, the full system was not yet formulated, but the initial test jig board stack creation process was explored. Once a test jig stack was created from the design files of the unit to test, then the test sequence for that unit could be determined. Once the test sequence is defined, it was envisioned that the test sequence could be either coded manually in a scripting language and loaded on the SD card provided on the generic test jig board or the defined tests could be loaded into a code generating application to generate a binary file that could be bootloaded onto the generic test jig, or could generate the test script automatically. Both of these system were however later abandoned when the test sequence was incorporated into the Java swing framework (described later in this chapter). For the prototype initially the option of using tests pre-compiled into the test jig binary files was used.

The initial process for creating a test jig stack is identical to what was defined in section 2.3.4.

3.2.4 Design Review

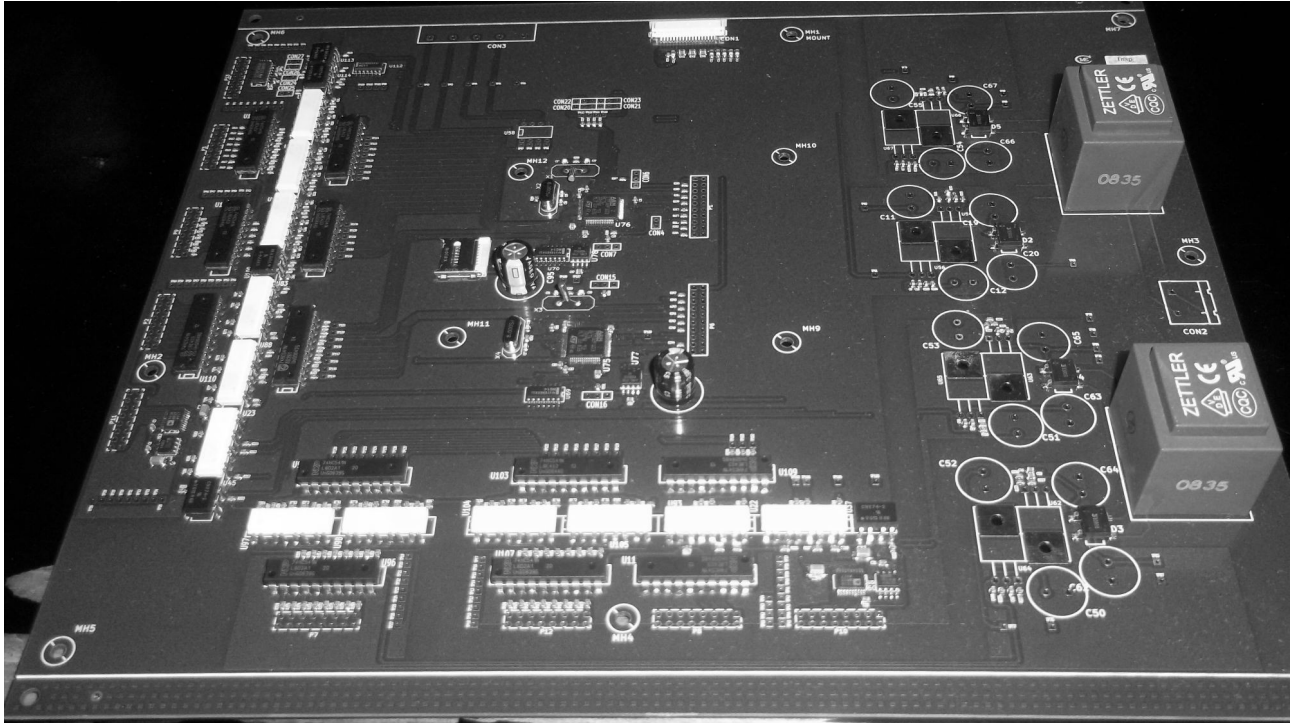


Illustration 21: Partially populated prototype board during test

Some severe problems were found with the initial prototype design, not least of which was the physical size of the test jig board at 295x235mm, as well as the physical size of the test bed of nails base at 800x400mm. There is also a very fixed interface between the two boards, making expandability very difficult. There were a massive number of changes suggested between the first prototype version and the first implementation of the test jig. These changes are discussed below:

- **change the isolation point in the design.**

Problem: In the prototype isolation was achieved using a very large number of opto couplers. The reason for the sheer number of optical isolators (and support logic chips for the directional selection) is the lack of expandability of boards – the test jig needs to make provision for the maximum number of inputs and outputs (or any combination thereof) for all conceivable designs. This sheer amount of I/O required necessitated the use of 2 processors as opposed to 1 (the processor chosen at the start of the process was one that was already stocked by UTIL LABS, and as such the benefits of economies of scale meant that it was in fact cheaper to use 2 of these processors, than 1 larger processor).

Solution: By shifting the isolation point to behind the RS232 comms port between

the PC and test board using iCoupler technology, a huge saving in logic pins could be made. This was the most significant change in the design, and was to be the preferred form of communication between the test jig and PC from then on out. This shifting of the isolation point also allowed for the use of the processor's on board peripherals.

- **Add scalability, shrink board.**

Problem: Trying to make provision for all eventualities by adding a lot of IO to the generic test board seemed like a good idea at the start of the prototype, but It was soon realised that there is no real gain in this approach.

Solution: make the boards smaller and therefore more scalable so the correct number of inputs, outputs and ADC pins could be chosen for each situation by adding the right number of boards.

- **Connection Standardisation.**

Problem: The risk of damaging the test jig unit, or some other unit by starting up into the incorrect state was deemed to be too high. The multiple interface points between the base and test bed of nails was a bad idea – its always better to keep the interface simple.

Solution: The decision was taken to standardize the interface connector between the generic test jig board, and the test jig bed of nails. The interface connector was standardised to the 40 pin connector used till the final version, as opposed to the smaller “single port” ribbon cables on the prototype.

- **Mechanical strain point moved.**

Problem: The mechanical strain that would be placed on the Base board when the UUT is pressed down would cause some flexing of the board, which could cause solder joints to crack and the test jig to become unreliable. using the Guide board just to keep the pins straight would place all of the strain onto the base board.

Solution: Test point placement was re-evaluated after the prototype. The placement scheme would be to print a silkscreen version of the outline of the UUT on the test jig bed of nails base, and then position the test points on the exact positions that they would be on the UUT. This allowed test pins to be added to an additional stopper layer in the stack where they could be fastened, such that the stopper board would take up any bending due to the force exerted onto the test pins, and the test pins could be joined to the base of the bed of nails via short

jumper wires. This soldering of wires between the test pins and the base of the bed of nails would nullify the mechanical strain placed on the board.

- **Removed SD card.**

Problem: The idea of issuing a test jig with an SD card seemed like a good idea, but the problem is that every time a new set of serial numbers needs to be issued from Util Labs to the factory, it would involve re-writing the card. This would mean that either a new SD card needs to be issued from UTIL LABS to the factory for each production run, or the serial numbers would need to first be loaded up to the test jig PC, and then sent down to the jig for storage. Additionally, the possibility of someone placing the wrong SD card into the wrong test jig, becomes problematic, especially since the initial idea was also to house the test scripts for the jig on the SD card as well. An advantage of having the serial keys and firmware on the SD card, was that it would be easier to have the test jig then program the UUT using a serial bootloader. This option however was not particularly attractive though, as all boards designed at UTIL LABS so far had exposed the JTAG port for programming rather than the serial port – and implementing a full JTAG interface on the test jig processor was deemed to bad idea since there was already an existing implementation on the test jig PC that could be modified slightly to work in the framework. Another problem with keeping all of the information on the SD card, is that it is a losable medium. If the SD card is swapped out during production runs (or between production runs) then all of the logging data would be lost.

Solution: The solution to the problem is to keep the serial numbers and test sequences with the JTAG implementation, which is on the PC. The initial idea, was to have a generic test jig, which could then be fitted with a specific bed of nails/SD card combination, but the SD card offers no real value. By removing the SD card, the need for a fat file system is nullified.

JTAG	Serial bootloader
Needs additional hardware to isolate the JTAG port.	Easy to isolate since the processor is already on the same ground potential
An implementation already exists that can be used in the PC application	Would need to be coded from scratch
No redesign required on previous product at UTIL LABS	Would need some product to be redesigned slightly
Needs only the firmware versions and PC to work	If implemented on the test jig, then the SD card would be needed to house the firmware due to its size on some of the processors
	Risk of losing SD card could stop the production run and badly compromise serial number security

Table 3: comparing JTAG and serial bootloader programming on the test jig

- **Single JTAG per test jig as opposed to JTAG scan chain.**

Problem: Since the logging, programming, and test script responsibilities were moved to the PC, The JTAG scan chain was investigated as a way of using a central PC to communicate with multiple jigs, but this poses the problem of synchronisation, as only 1 device can be programmed at a time with the JTAG interface, so each of the tests would have to be aware of the status of the other tests, which is overly complex. Additionally by using the JTAG scan chain, the wiring between test jigs become overly complex, and limits the ability of the jigs to be moved around and reconfigured. Isolation between the different jigs is also of concern when running with the scan chain as it is simple to isolate the RS 485 bus, but due to the speed it runs at, isolating the JTAG scan chain become significantly more complex.

Solution: It was with this in mind, that the RS 485 and scan chain bus were dropped in preference to using low cost pcs that each have a JTAG key device and RS232 assigned to it. This simplifies the logging, programming and all communication problems between jigs, and the cost was deemed to be acceptable for the gain offered.

- **Electrical Protection.**

Problem: Electrical protection was a problem in the prototype, as the neither the

CMOS inputs or the ADC inputs were protected.

Solution: With this in mind zener diodes and series resistors are added to all input lines.

- **Test Sequence and installation**

Problem: Initially it was proposed to use scripts to set what would be tested on each test jig, However, with all of the changes proposed above, the test script implementation seemed to be a less attractive solution as it would require the operator to be aware of the test scripts on installation of the jig and he would need to ensure that the correct script is run on the correct jig (if an output that is normally connected to mains for example was switched at the wrong time it could cause massive damage).

Solution: A better solution is proposed by allowing all of the possible tests to be housed in a single jar file written in Java and using the spring framework add a GUI to it. This way, by adding a few select lines to the bed of nails base, the appropriate jig behaviour can be chosen in software from a list of different test jigs automatically. This way, there is no need by the operator to install the correct script, as It will already be a part of the framework that is required to run the jig. Configurable parts of the system can be added to the .prop resources file. An additional advantage to having this single jar file approach, is that in the event of a new test jig becoming available, if the jig select lines do not conform to a known jig for the version of the jar file on the test jig PC, then the program can display an error instead of running ahead preventing all damage to the jig from incorrect software set up.